Object Oriented Analysis and Design Using the UML Version 4.2

Module 7C: Use-Case Analysis

Copyright © 1998-1999 Rational Software, all rights reserved

Rational
unifying software teams

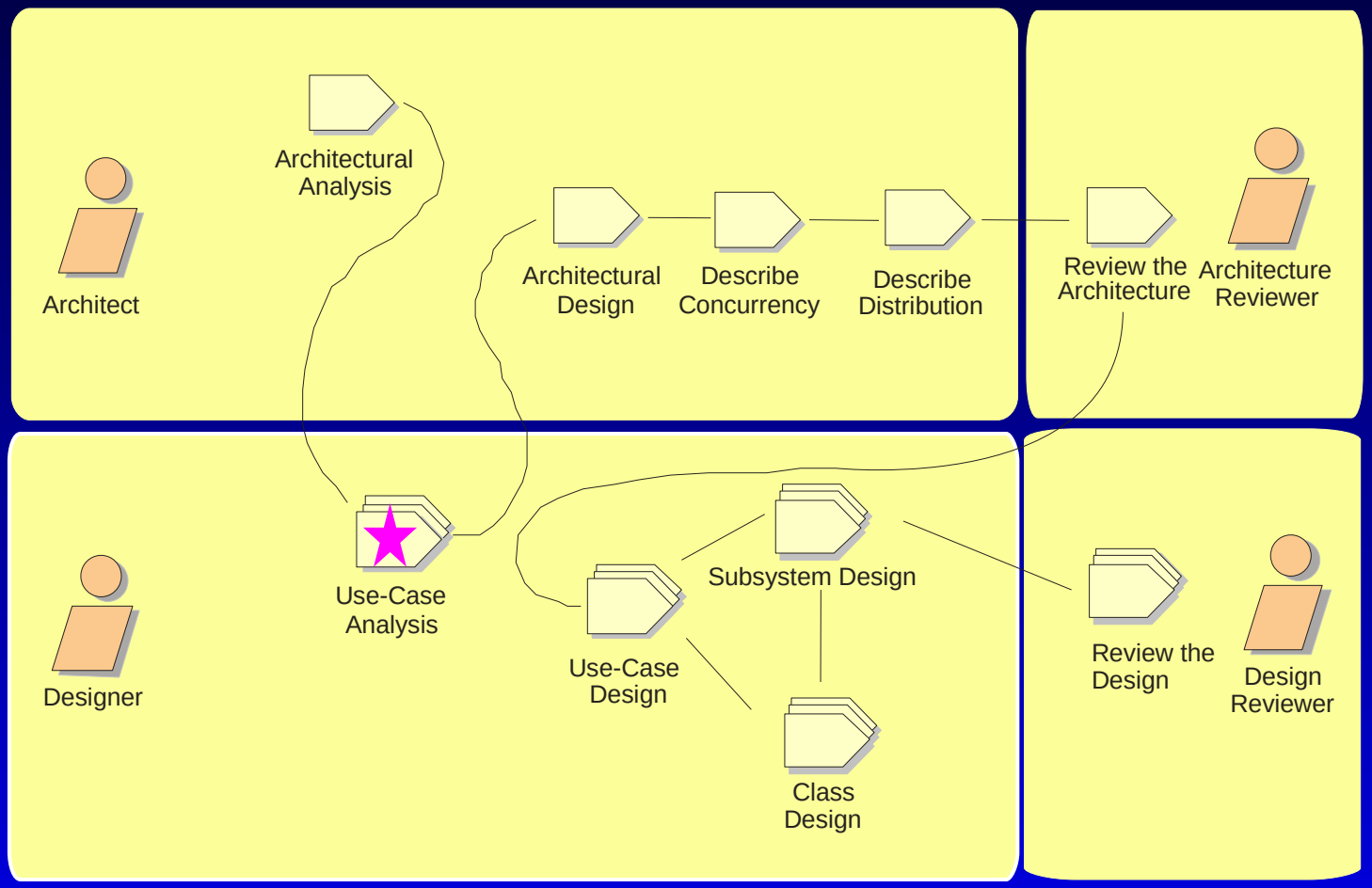
Objectives: Use-Case Analysis

- Understand the purpose of Use-Case Analysis and where in the lifecycle it is performed
- Identify the classes which perform a use-case flow of events
- Distribute the use-case behavior to those classes, identifying responsibilities of the classes
- Develop use-case realizations that model the collaborations between instances of the identified classes

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

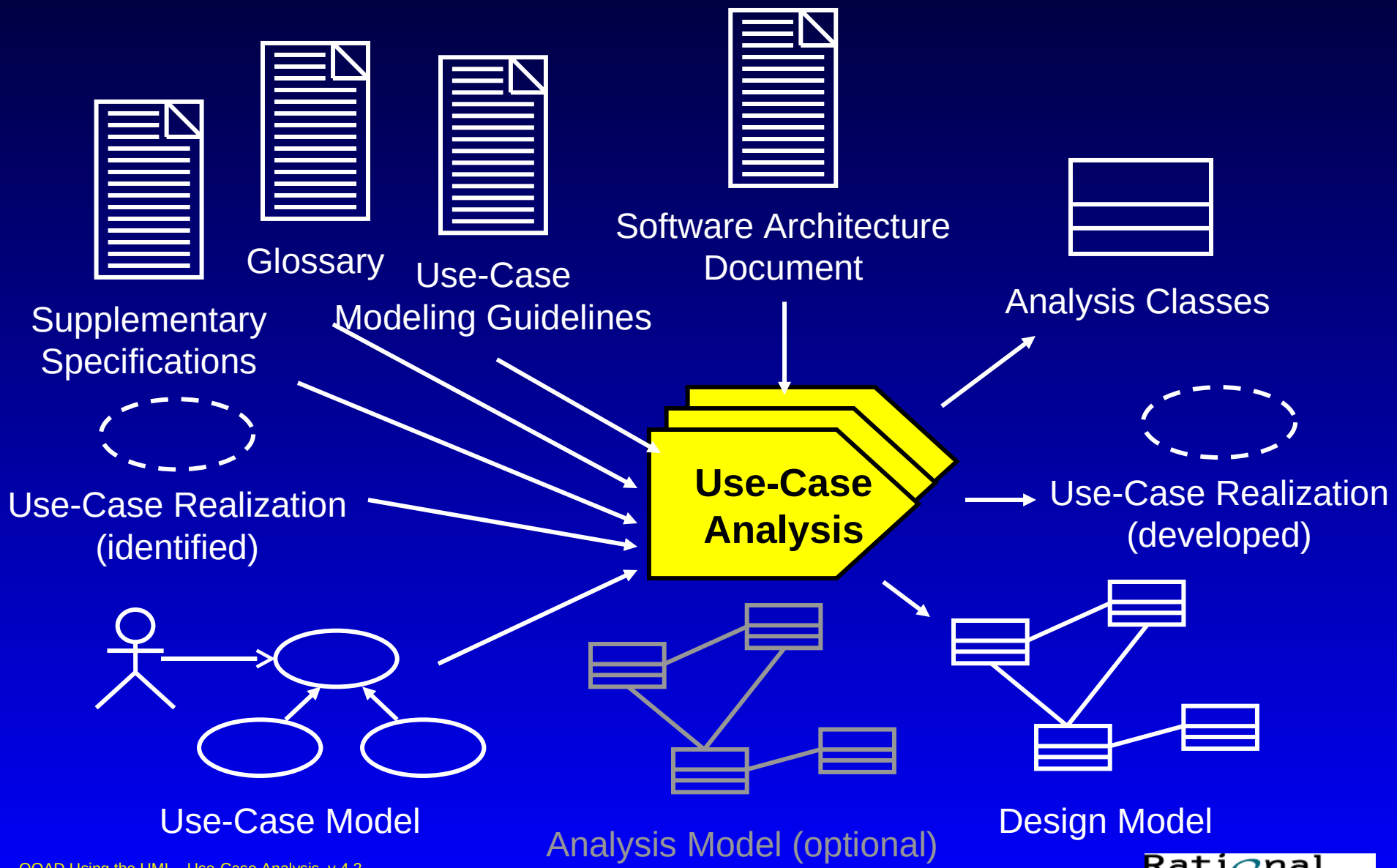
Use-Case Analysis in Context



Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Use-Case Analysis Overview



Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Use-Case Analysis Steps

- Supplement the Use-Case Description
- For each use-case realization
  - Find Classes from Use-Case Behavior
  - Distribute Use-Case Behavior to Classes
- For each resulting analysis class
  - Describe Responsibilities
  - Describe Attributes and Associations
  - Qualify Analysis Mechanisms
- Unify Analysis Classes
- Checkpoints

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

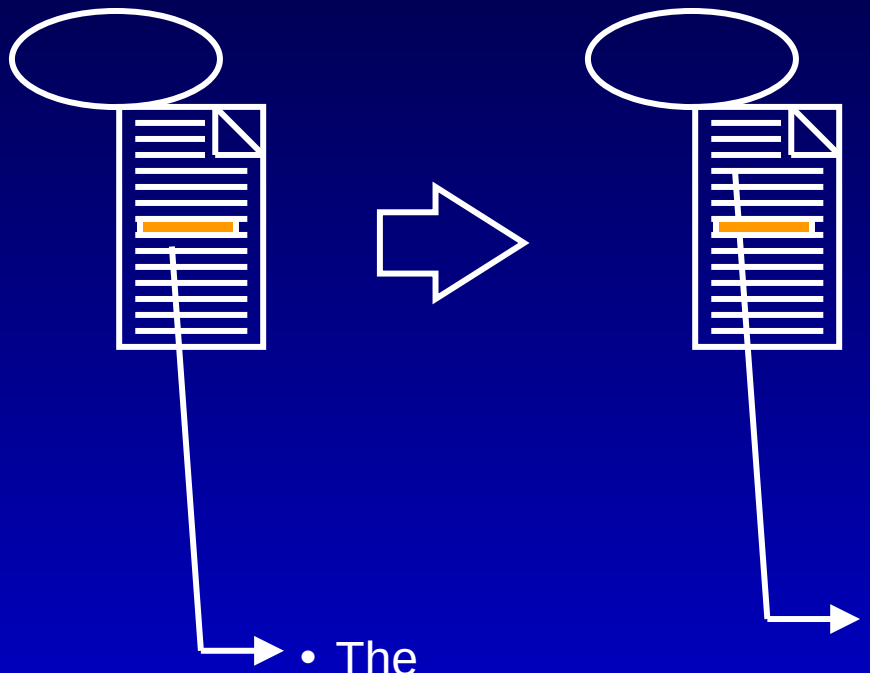
Use-Case Analysis Steps

- Supplement the Use-Case Description
- For each use-case realization
  - Find Classes from Use-Case Behavior
  - Distribute Use-Case Behavior to Classes
- For each resulting analysis class
  - Describe Responsibilities
  - Describe Attributes and Associations
  - Qualify Analysis Mechanisms
- Unify Analysis Classes
- Checkpoints

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Supplement the Use-Case Description



- The system displays a list of course offerings.

- The system retrieves and displays a list of current course offerings from the course catalog legacy database.

Copyright © 1998-1999 Rational Software, all rights reserved

Rational
unifying software teams

Use-Case Analysis Steps

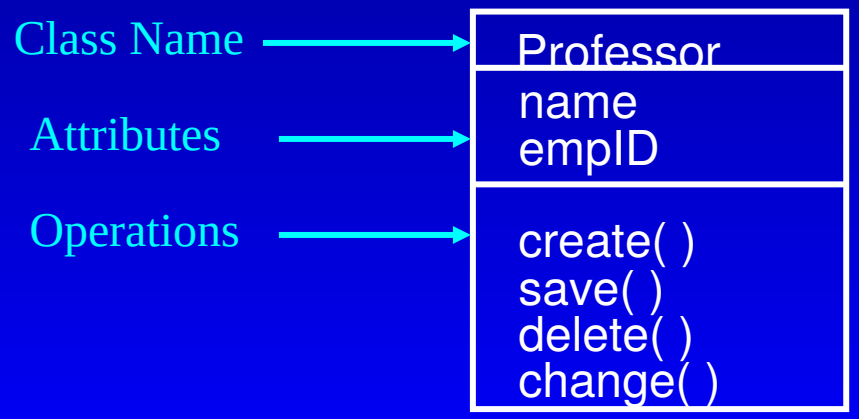
- Supplement the Use-Case Description
- For each use-case realization
  - Find Classes from Use-Case Behavior
  - Distribute Use-Case Behavior to Classes
- For each resulting analysis class
  - Describe Responsibilities
  - Describe Attributes and Associations
  - Qualify Analysis Mechanisms
- Unify Analysis Classes
- Checkpoints

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Review: Class

- An abstraction
- Describes a group of objects with common:
  - Properties (attributes)
  - Behavior (operations)
  - Relationships
  - Semantics



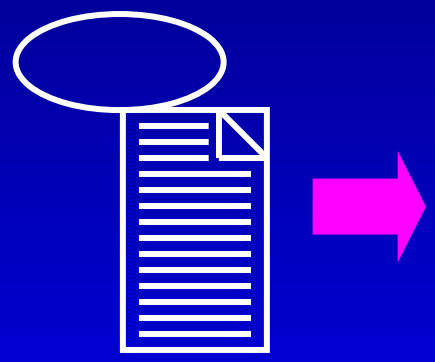
Copyright © 1998-1999 Rational Software, all rights reserved

Rational
unifying software teams

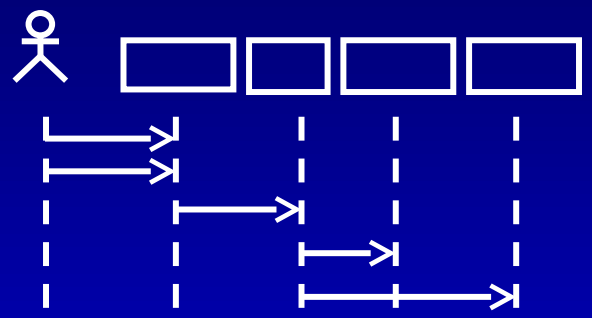
Review: Use-Case Realization

Use-Case Model *Design Model*

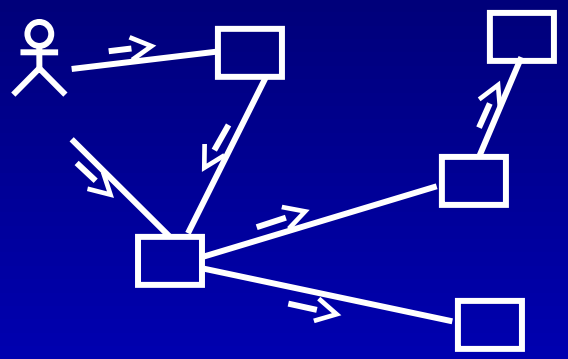
Use Case Use-Case Realization



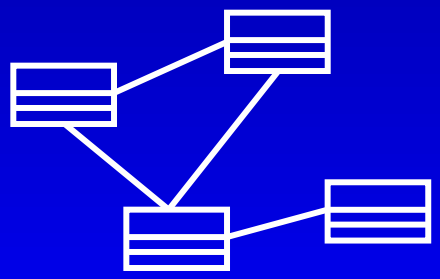
Use Case



Sequence Diagrams



Collaboration Diagrams



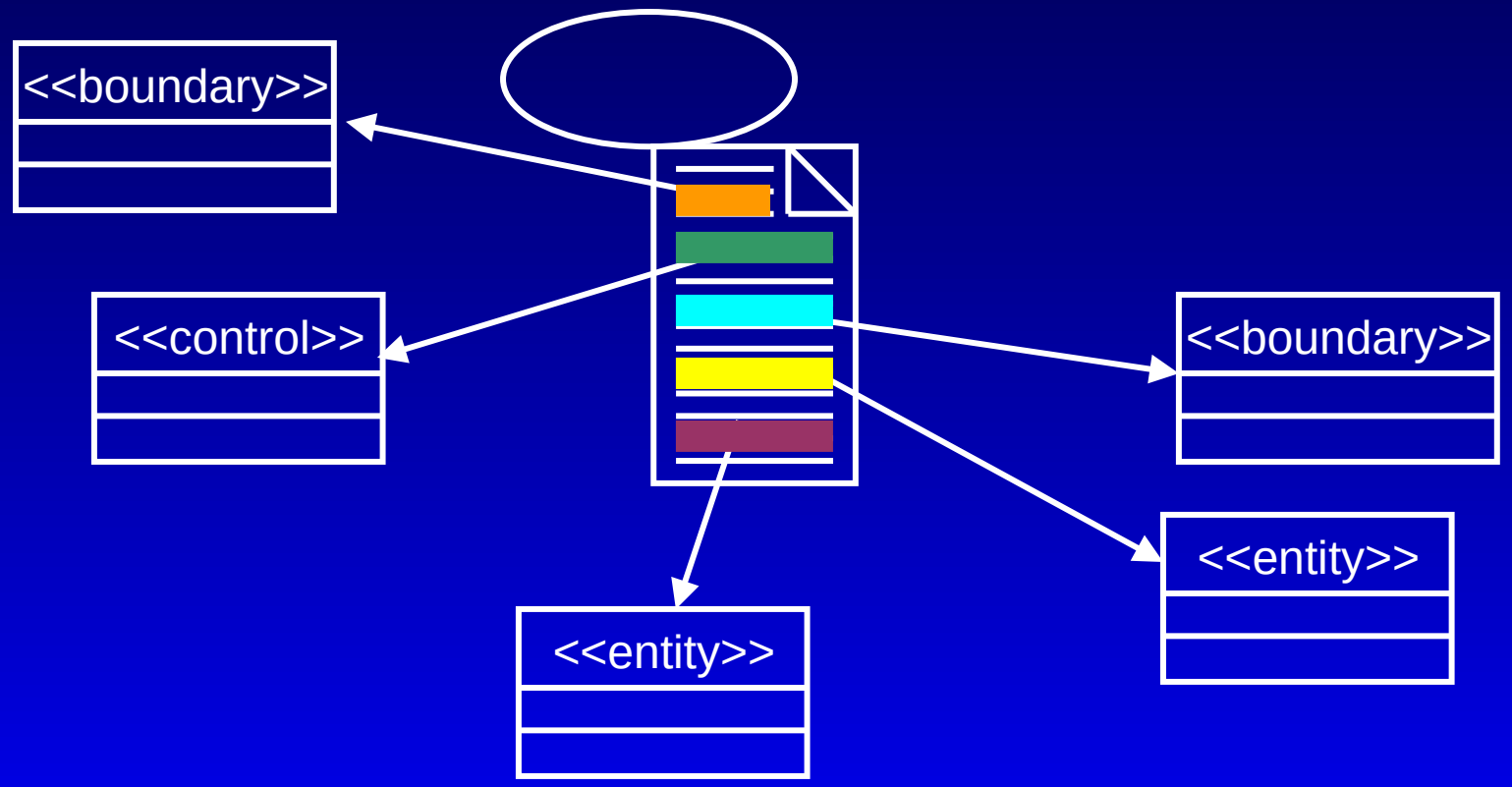
Class Diagrams

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Find Classes From Use-Case Behavior

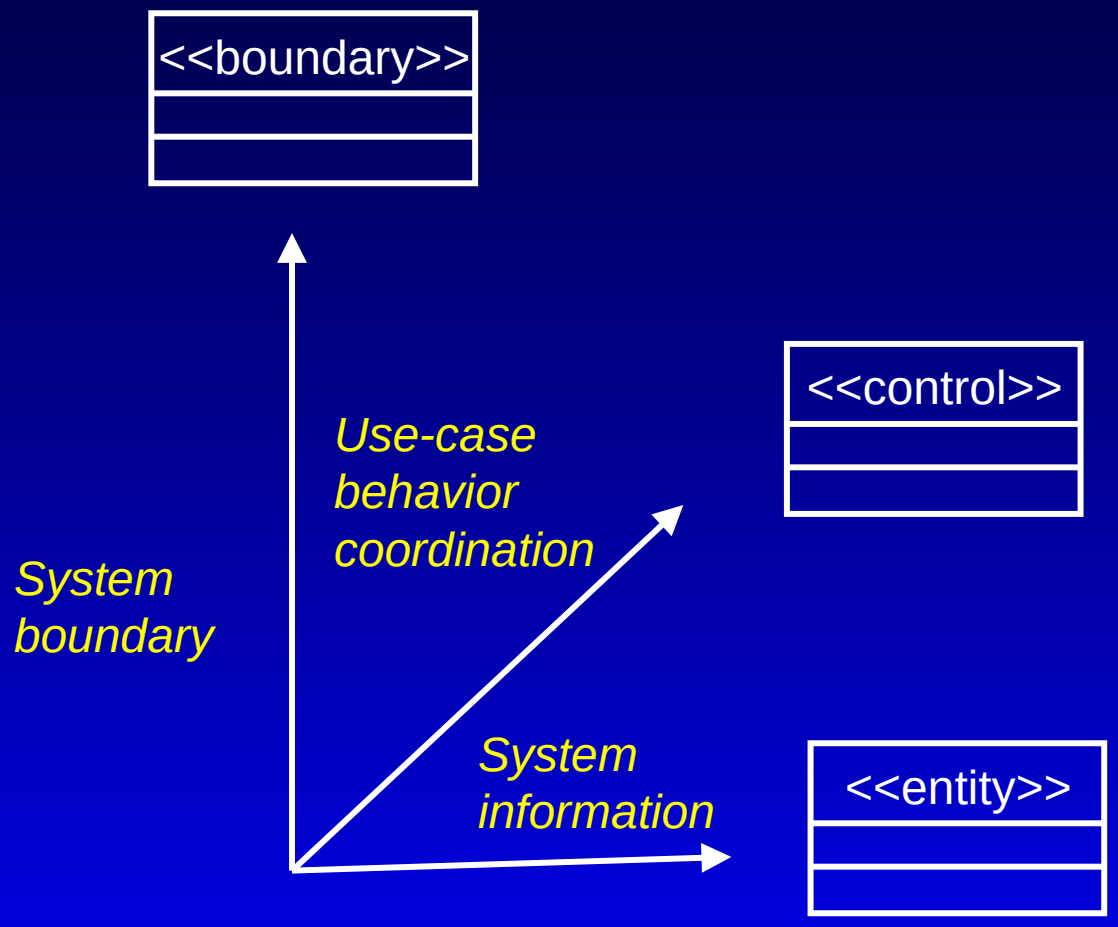
- The complete behavior of a use case has to be distributed to analysis classes



Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

What is an Analysis Class?



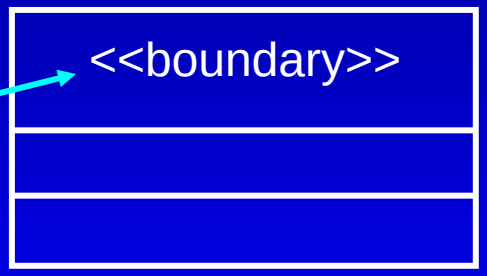
Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

What is a Boundary Class?

- Intermediates the interface to something outside the system
- Several Types
  - User interface classes
  - System interface classes
  - Device interface classes
- *One boundary class per actor/use case pair*

Analysis class stereotype

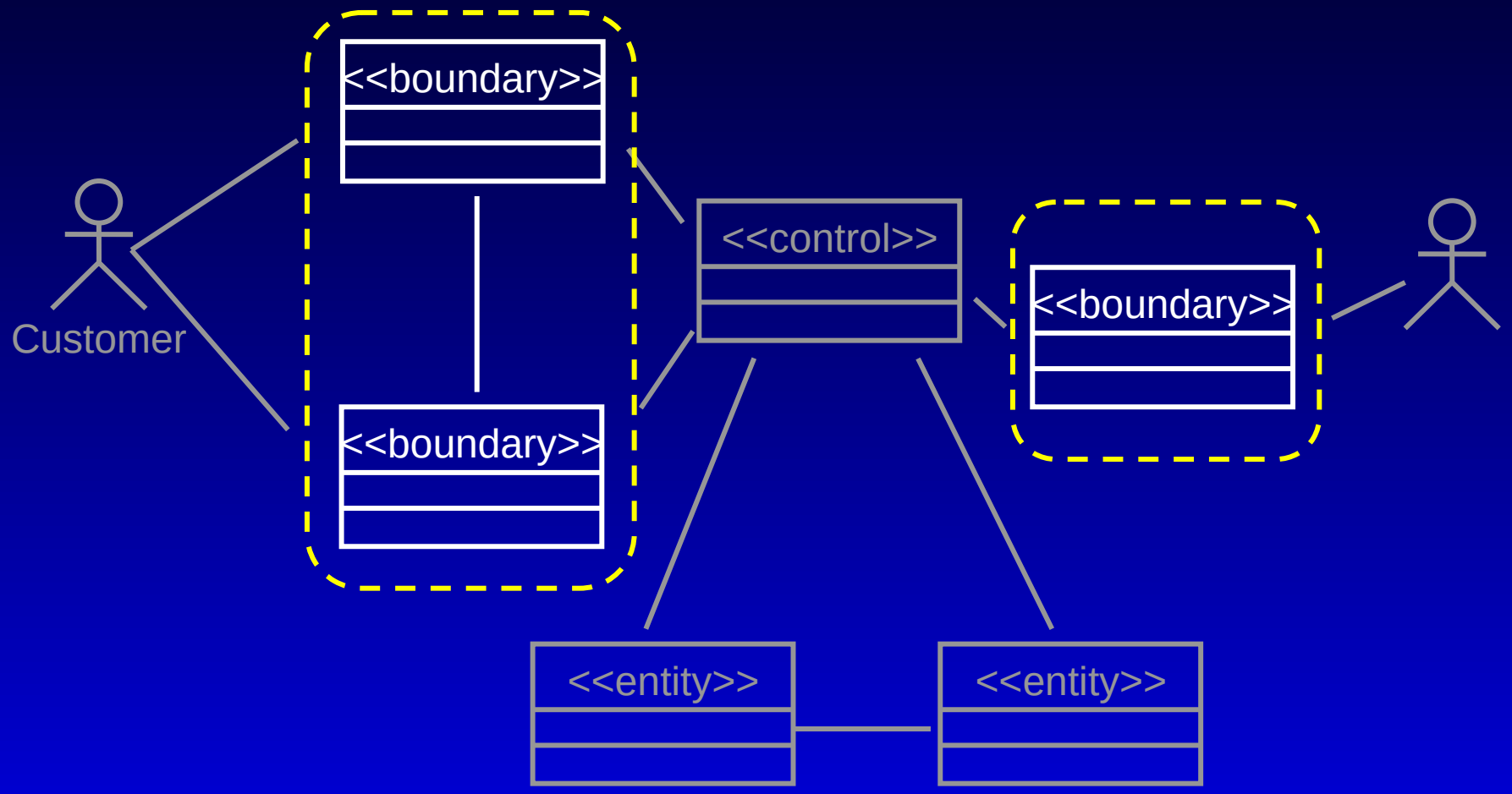


Environment Dependent

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

The Role of a Boundary Class



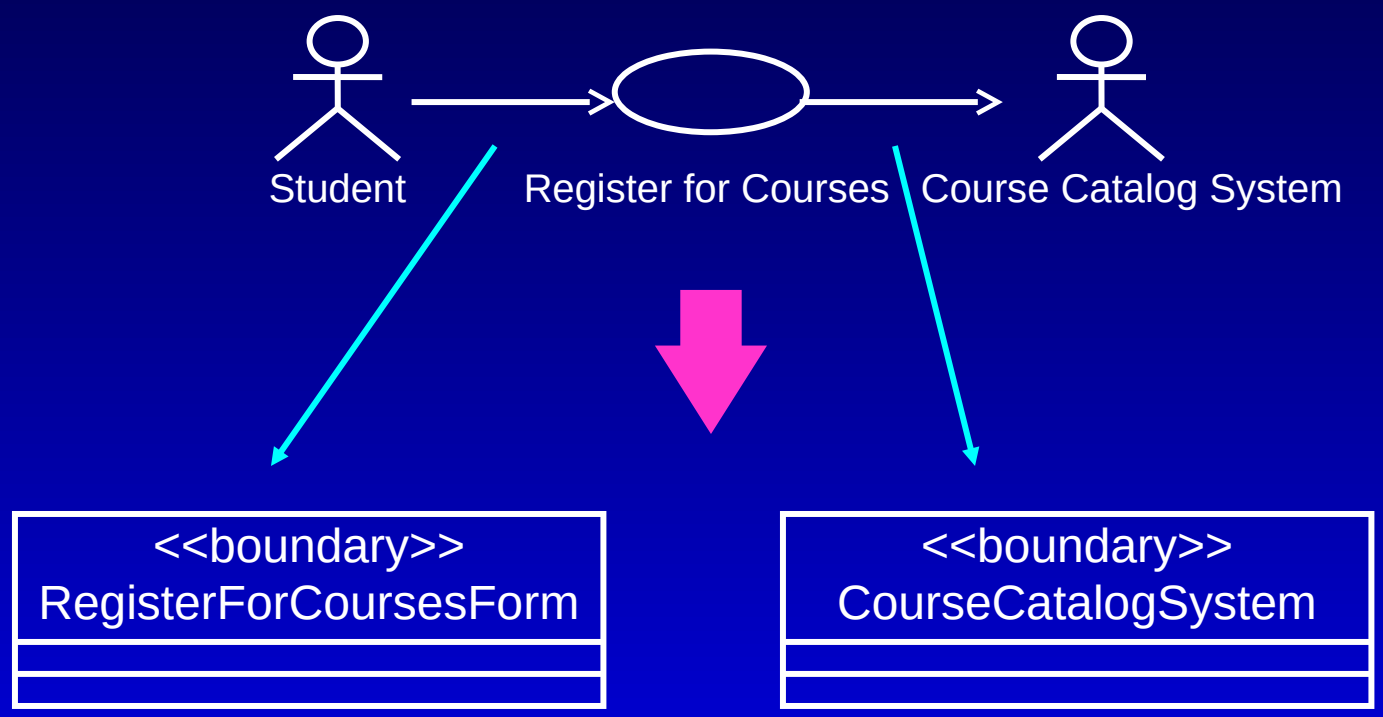
Model interaction between the system and its environment

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Example: Finding Boundary Classes

- One boundary class per actor/use case pair



Copyright © 1998-1999 Rational Software, all rights reserved

Guidelines: Boundary Class

- User Interface Classes
  - Concentrate on what information is presented to the user
  - Do NOT concentrate on the UI details
- System and Device Interface Classes
  - Concentrate on what protocols must be defined
  - Do NOT concentrate on how the protocols will be implemented

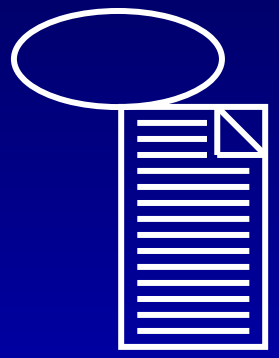
Concentrate on the responsibilities, not the details!

Copyright © 1998-1999 Rational Software, all rights reserved

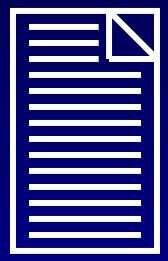
Rational unifying software teams

What is an Entity Class?

- Key abstractions of the system

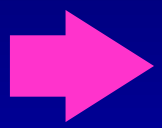


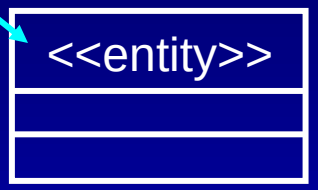
Use Case

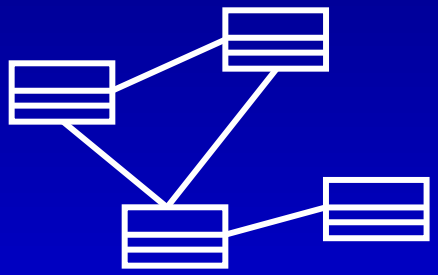


Glossary

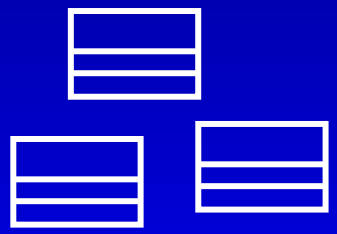
Analysis class stereotype







Business-Domain Model

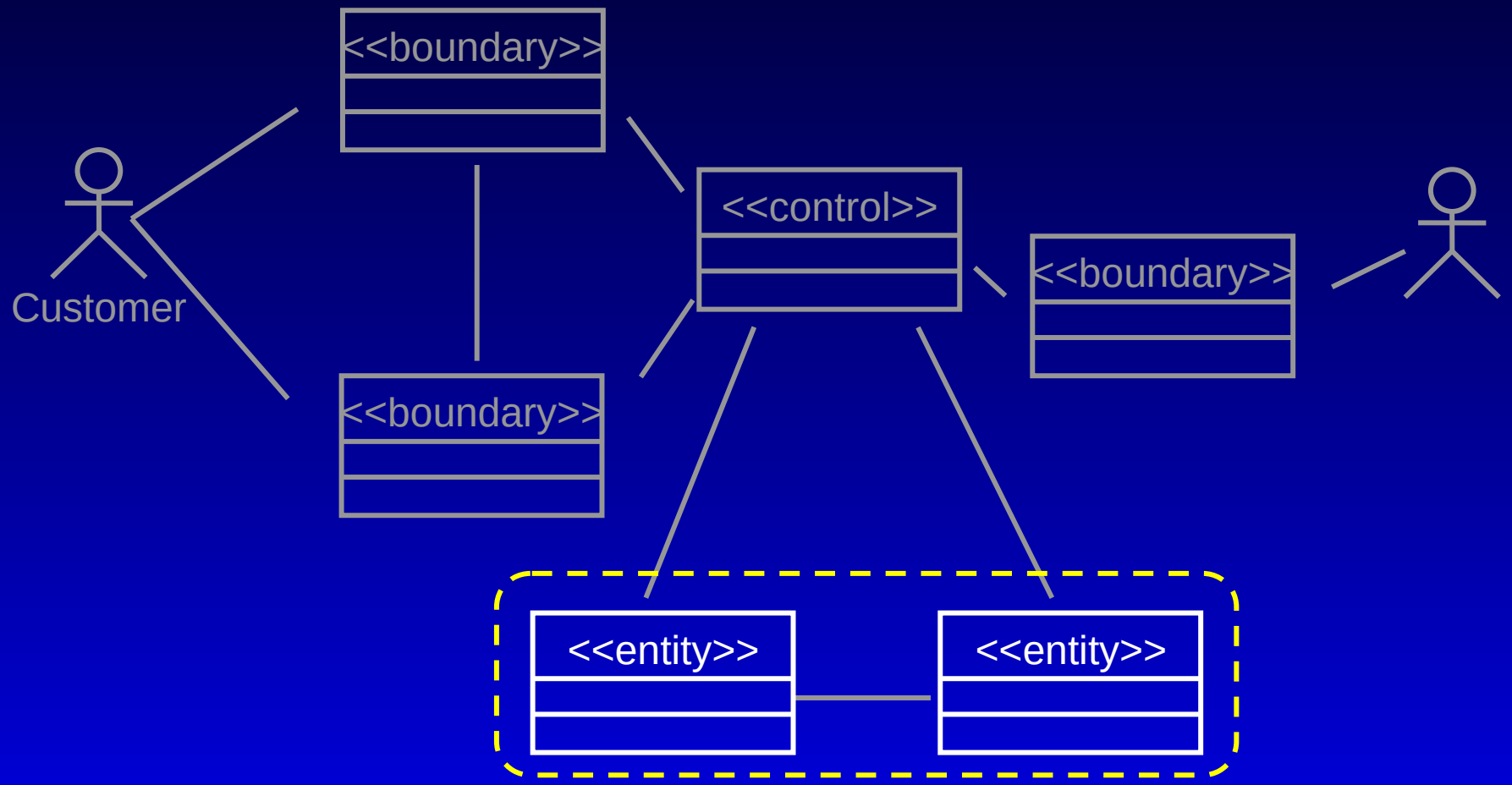


Architectural Analysis Abstractions

Environment Independent

Copyright © 1998-1999 Rational Software, all rights reserved

The Role of an Entity Class



Store and manage information in the system

Copyright © 1998-1999 Rational Software, all rights reserved

Rational
unifying software teams

Example: Finding Entity Classes

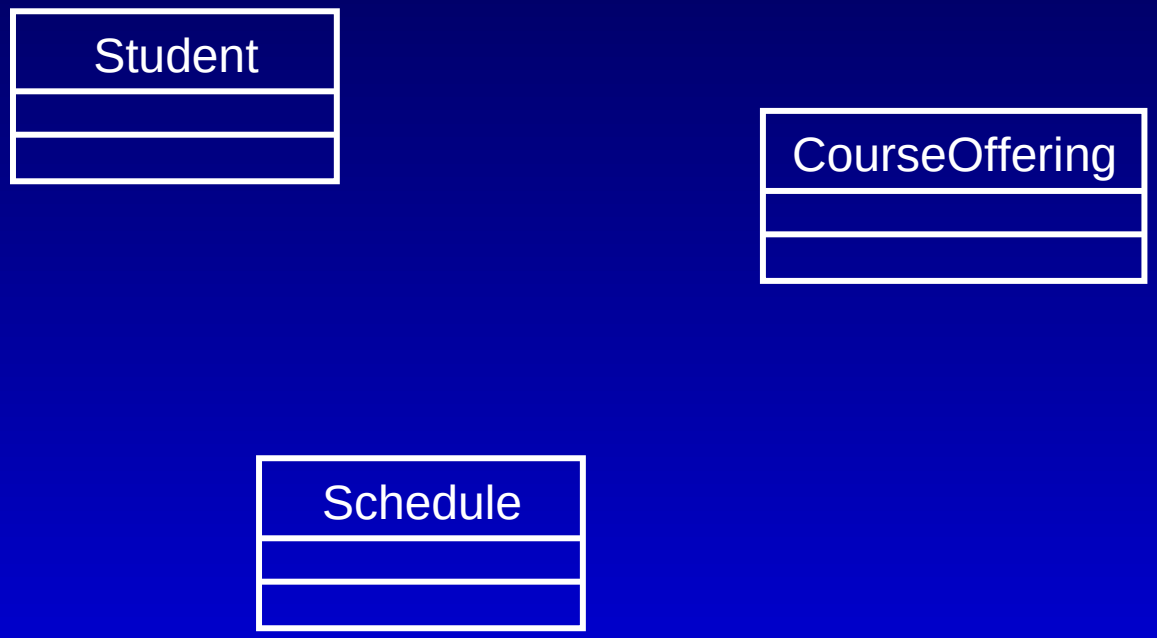
- Use use-case flow of events as input
- Key abstractions of the use case
- Traditional, filtering nouns approach
  - Underline noun clauses in the use-case flow of events
  - Remove redundant candidates
  - Remove vague candidates
  - Remove actors (out of scope)
  - Remove implementation constructs
  - Remove attributes (save for later)
  - Remove operations

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Example: Candidate Entity Classes

- Register for Courses (Create Schedule)

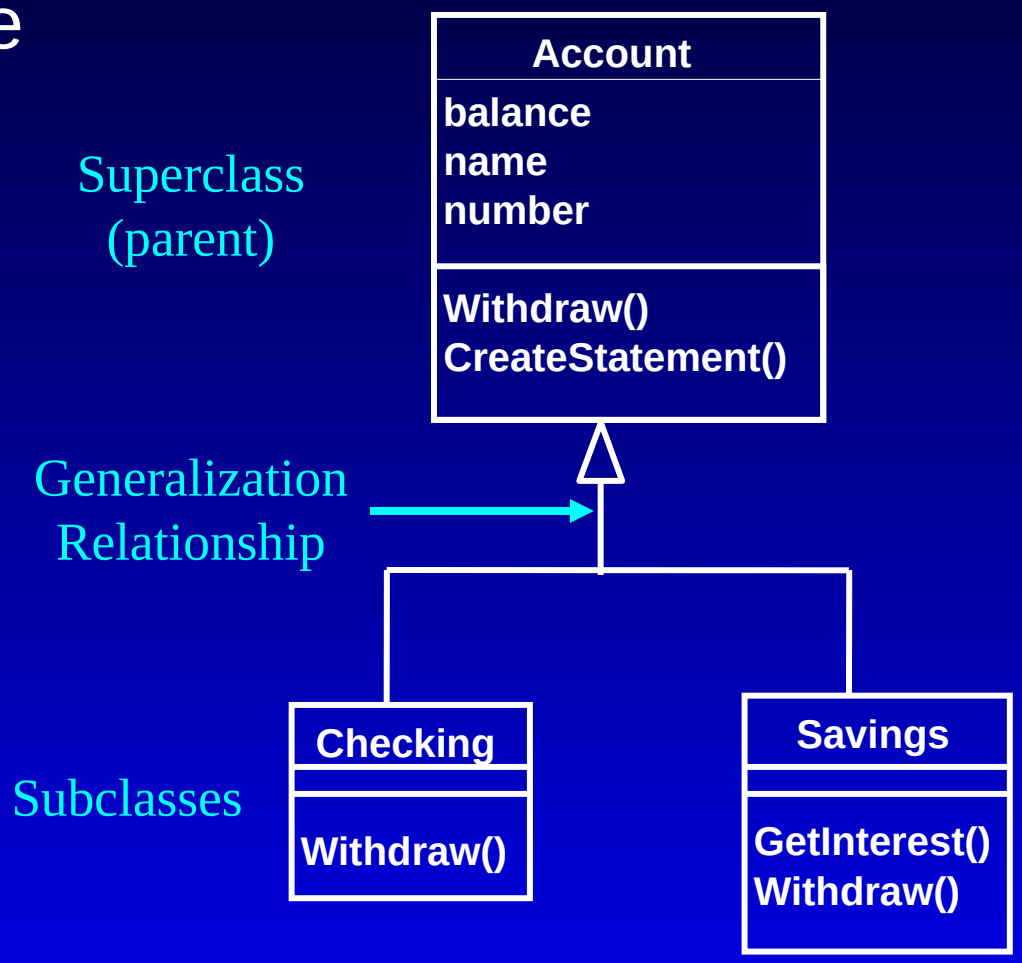


Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Review: Generalization

- One class shares the structure and/or behavior of one or more classes
- "Is-a-kind of" relationship
- In analysis, use sparingly

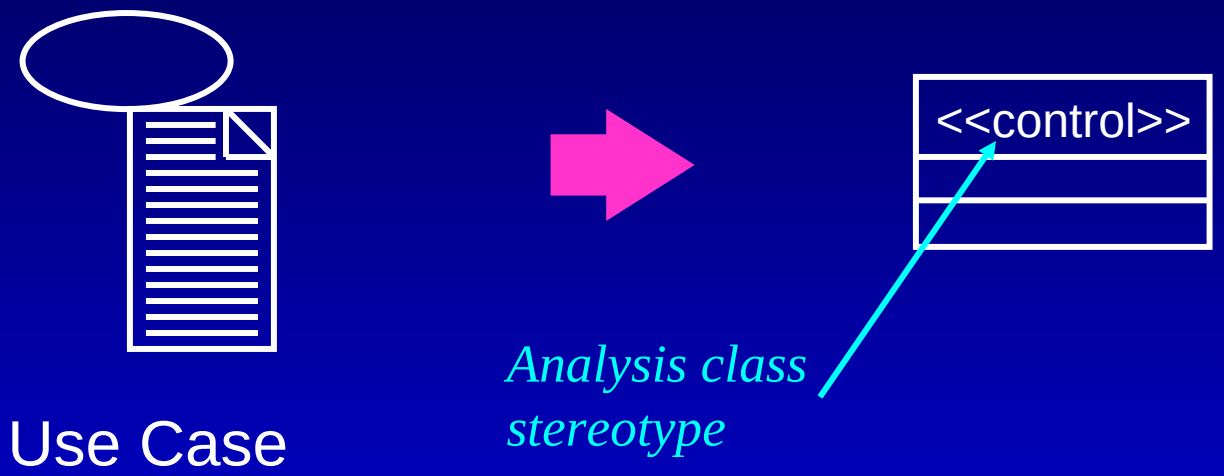


Copyright © 1998-1999 Rational Software, all rights reserved

Rational
unifying software teams

What is a Control Class?

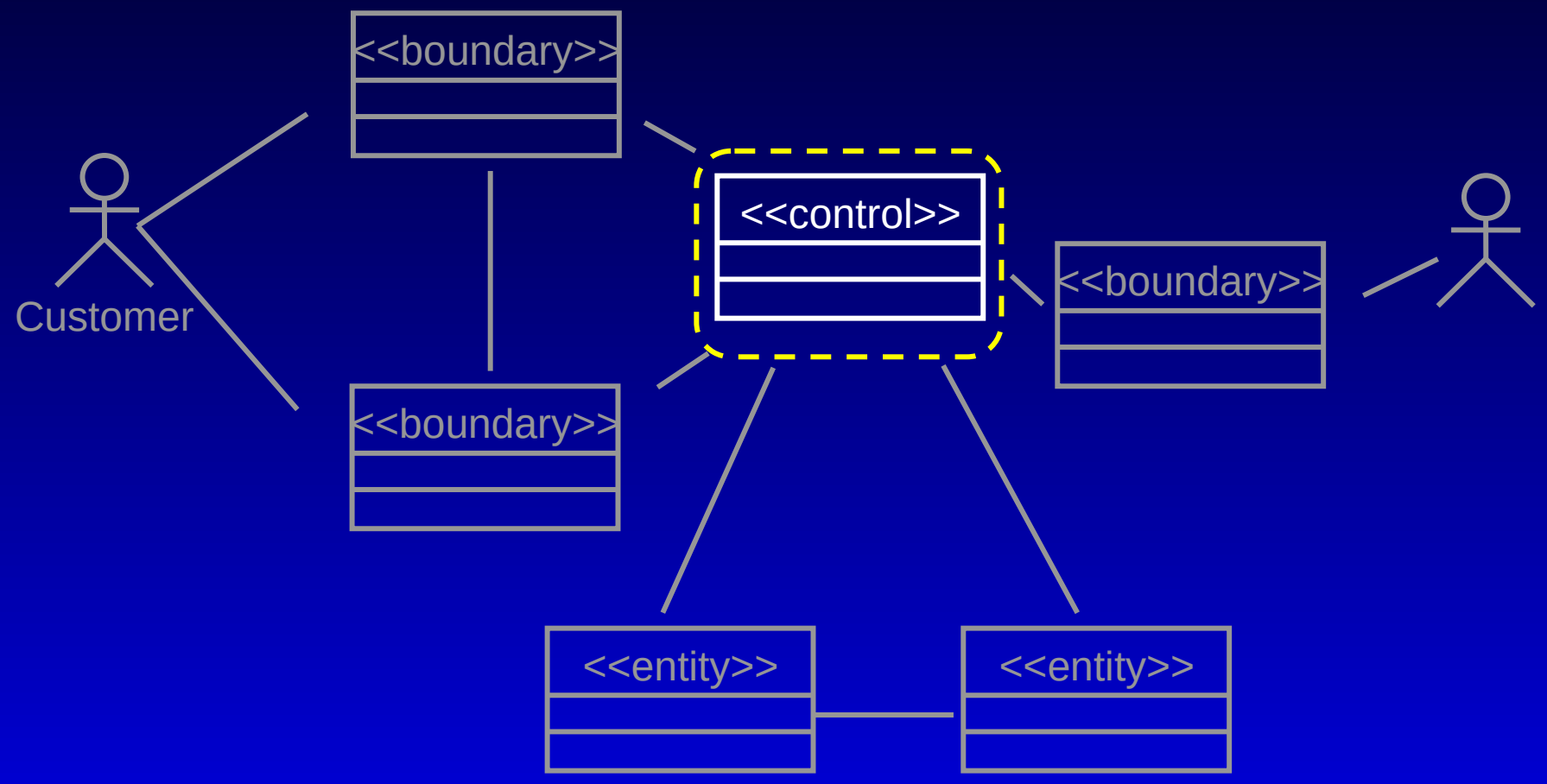
- Use-case behavior coordinator
- *One control class per use case*



Use-case dependent, Environment independent

Copyright © 1998-1999 Rational Software, all rights reserved

The Role of a Control Class



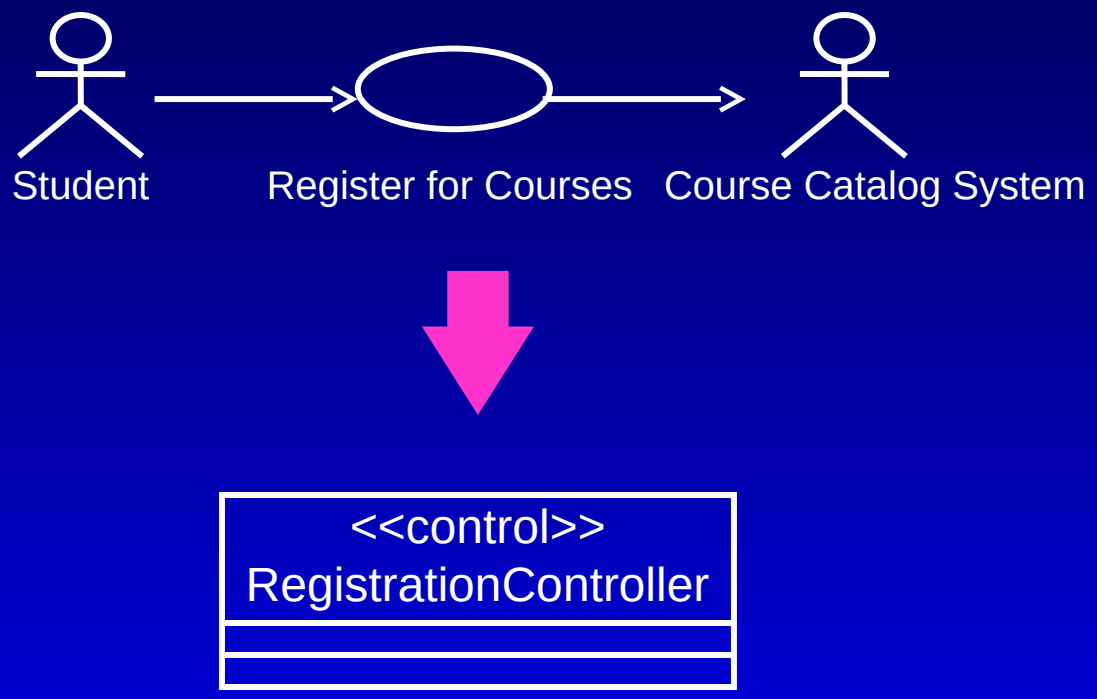
Coordinate the use-case behavior

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

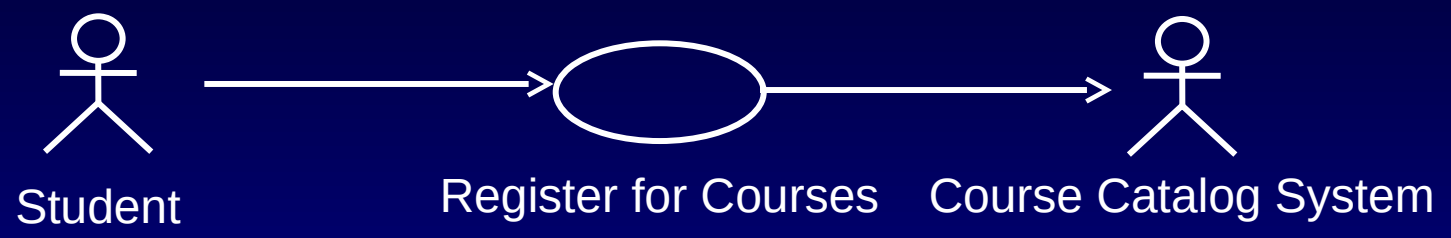
Example: Finding Control Classes

- One control class per use case



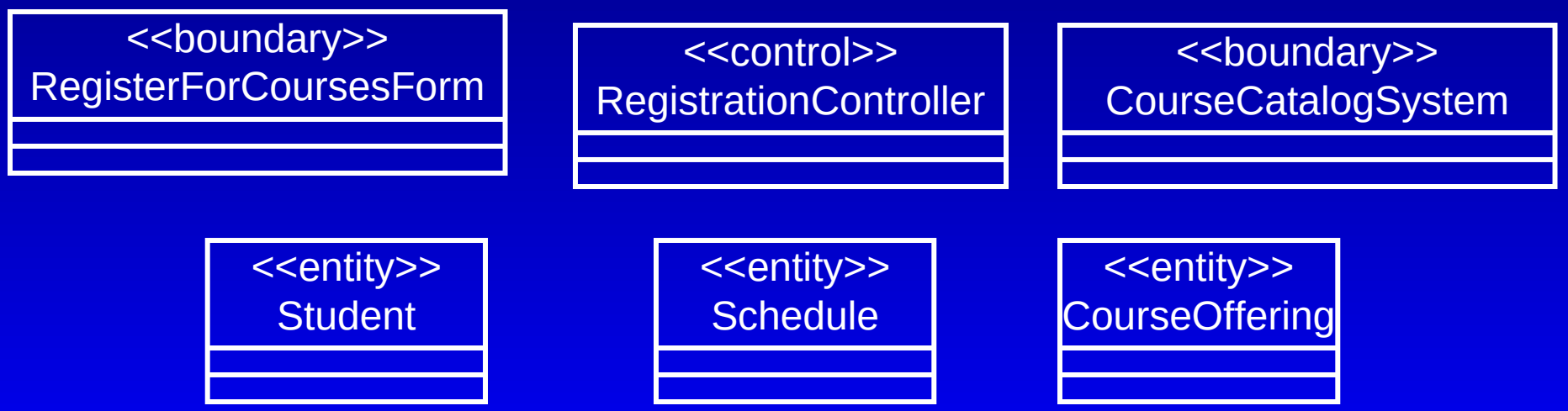
Copyright © 1998-1999 Rational Software, all rights reserved

Example: Summary: Analysis Classes



Use-Case Model

Design Model



Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Use-Case Analysis Steps

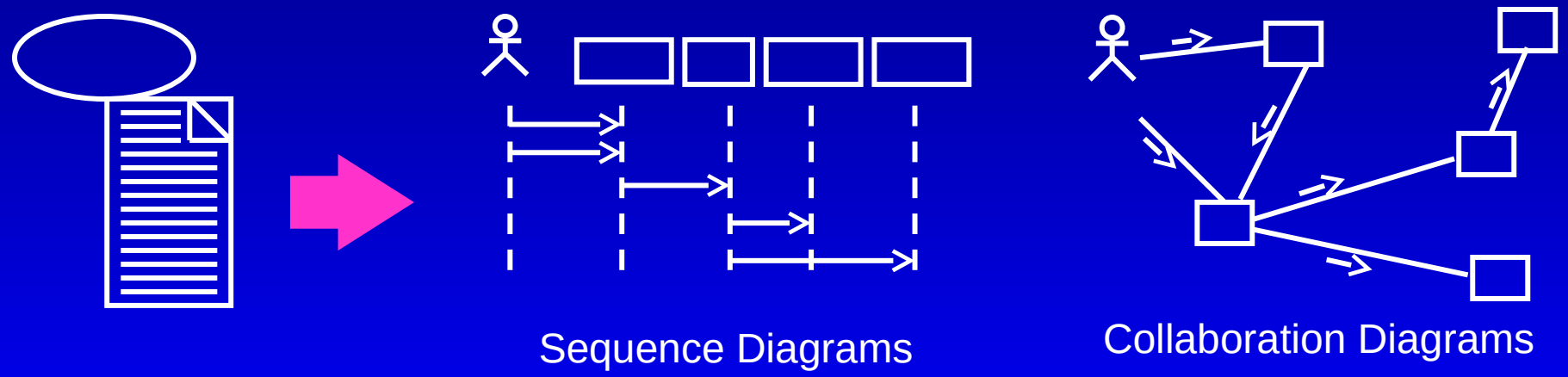
- Supplement the Use-Case Descriptions
- For each use-case realization
  - Find Classes from Use-Case Behavior
  - ★ Distribute Use-Case Behavior to Classes
- For each resulting analysis class
  - Describe Responsibilities
  - Describe Attributes and Associations
  - Qualify Analysis Mechanisms
- Unify Analysis Classes
- Checkpoints

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Distribute Use-Case Behavior to Classes

- For each use-case flow of events:
  - Identify analysis classes
  - Allocate use-case responsibilities to analysis classes
  - Model analysis class interactions in interaction diagrams



Use Case

Use-Case Realization

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Guidelines: Allocating Responsibilities to Classes

- Use analysis class stereotypes as a guide
  - Boundary Classes
    - Behavior that involves communication with an actor
  - Entity Classes
    - Behavior that involves the data encapsulated within the abstraction
  - Control Classes
    - Behavior specific to a use case or part of a very important flow of events

(continued)

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

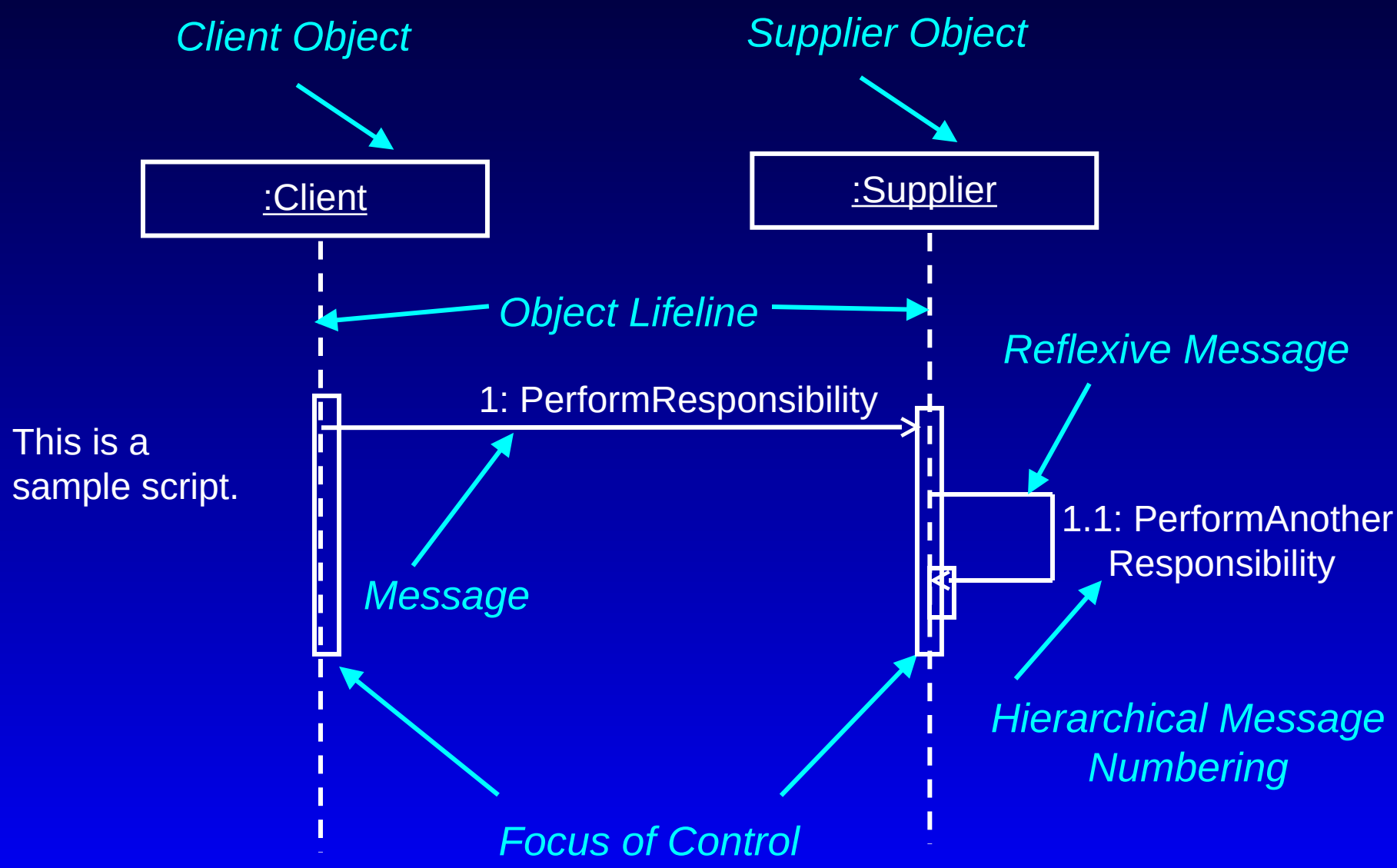
Guidelines: Allocating Responsibilities to Classes (cont.)

- Who has the data needed to perform the responsibility?
  - One class has the data, put the responsibility with the data
  - Multiple classes have the data:
    - Put the responsibility with one class and add a relationship to the other
    - Create a new class, put the responsibility in the new class, and add relationships to classes needed to perform the responsibility
    - Put the responsibility in the control class, and add relationships to classes needed to perform the responsibility

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

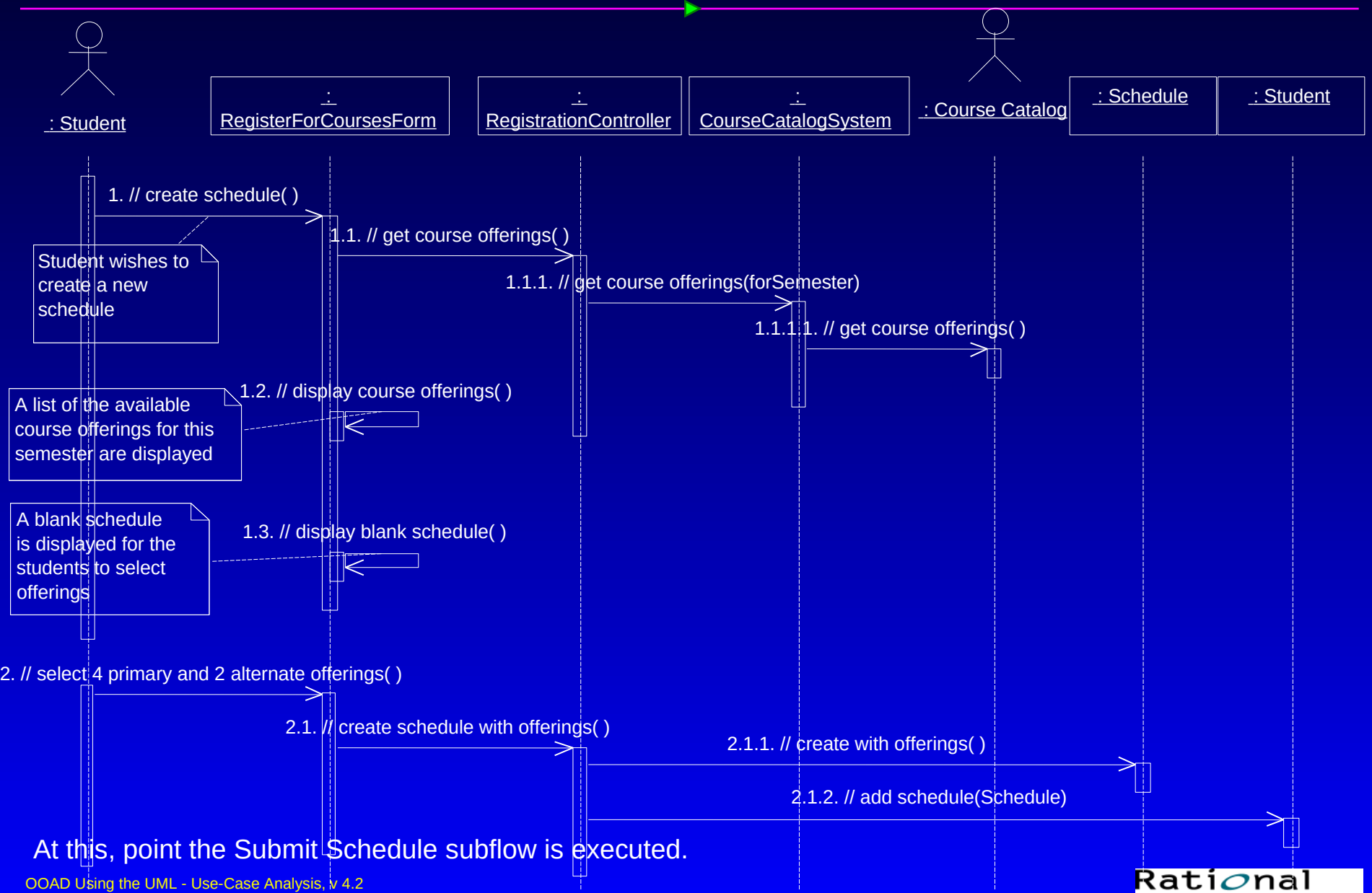
The Anatomy of Sequence Diagrams



Copyright © 1998-1999 Rational Software, all rights reserved

Rational
unifying software teams

Example: Sequence Diagram

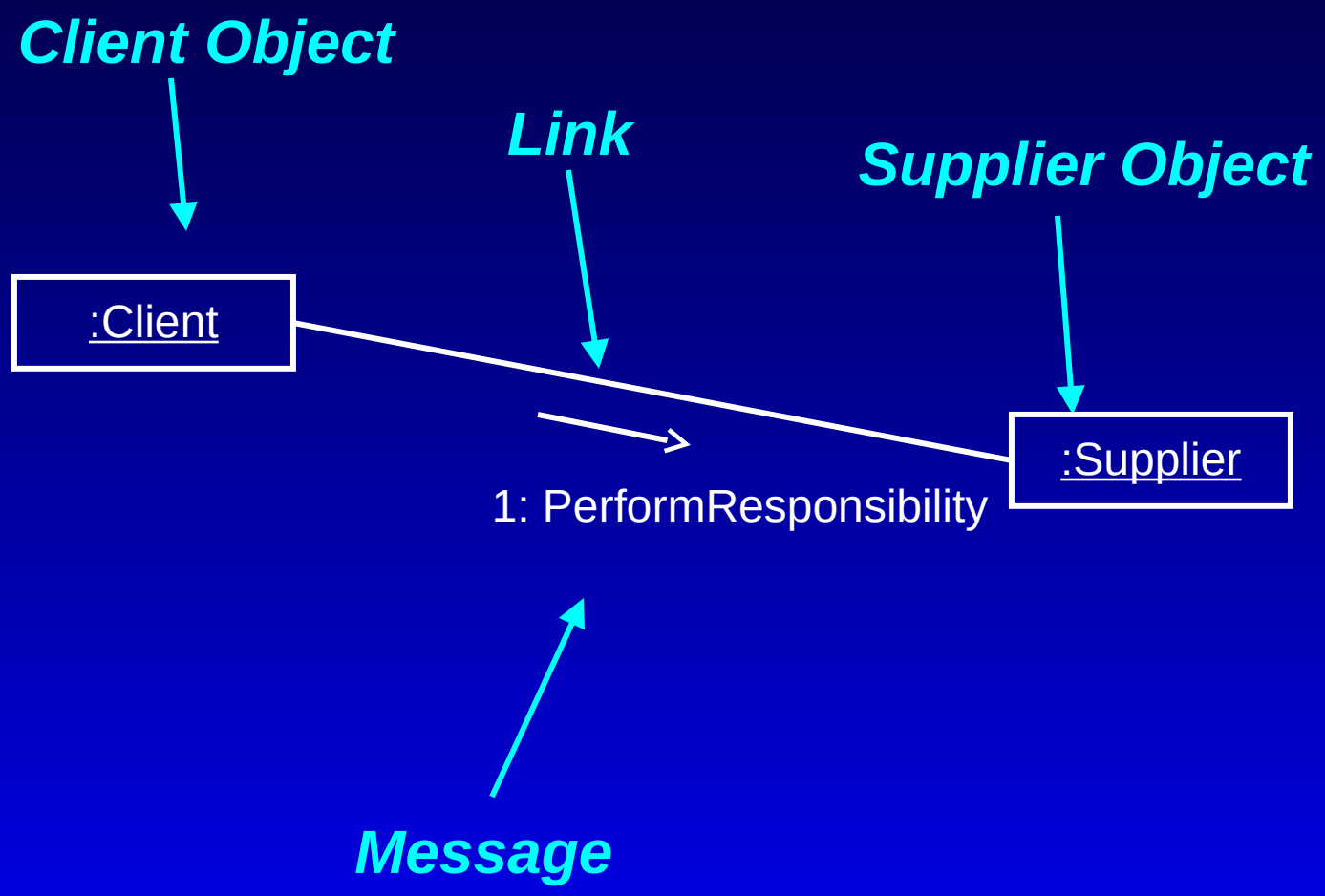


At this, point the Submit Schedule subflow is executed.

Copyright © 1998-1999 Rational Software, all rights reserved

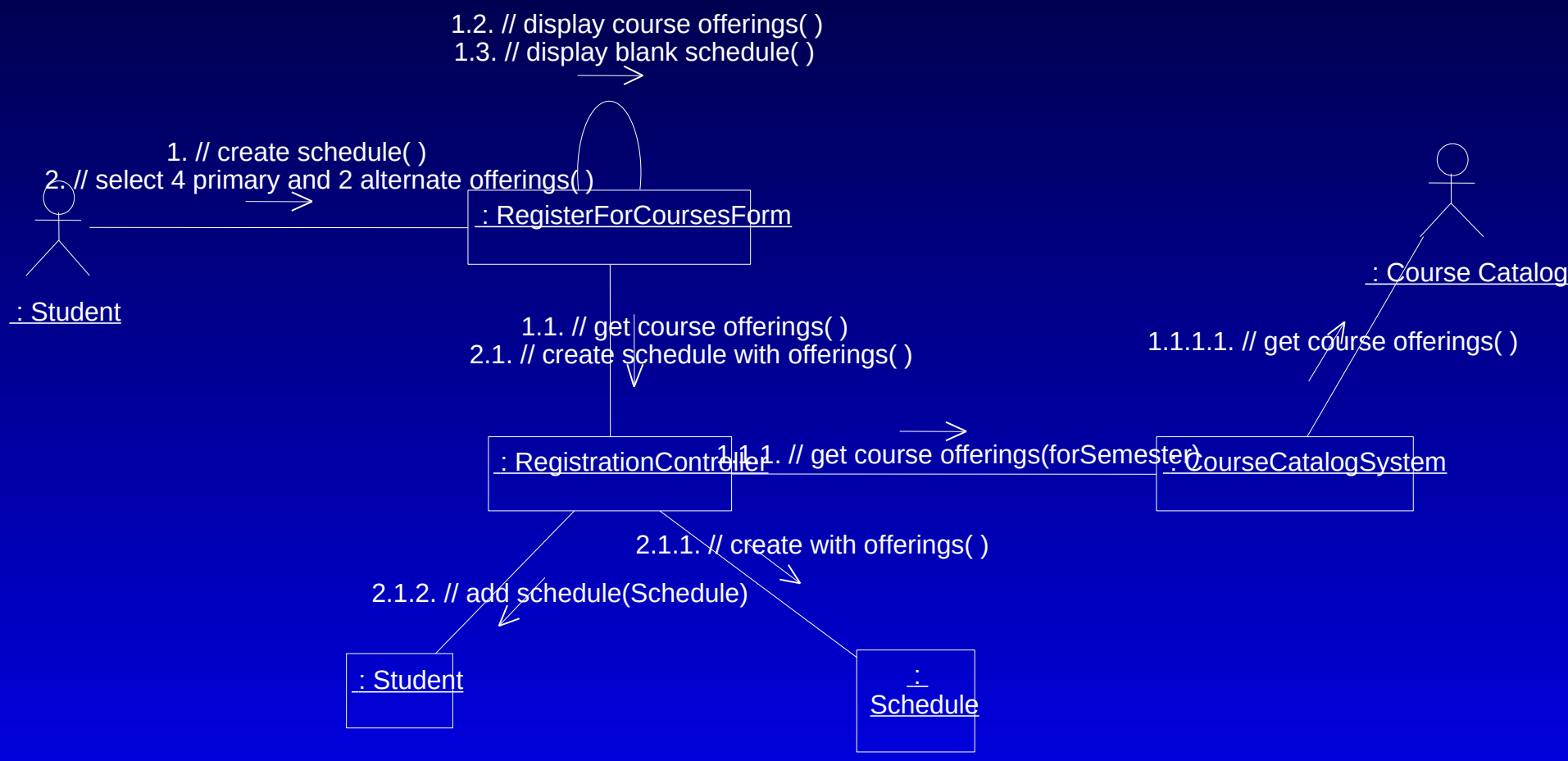
Rational unifying software teams

The Anatomy of Collaboration Diagrams



Copyright © 1998-1999 Rational Software, all rights reserved

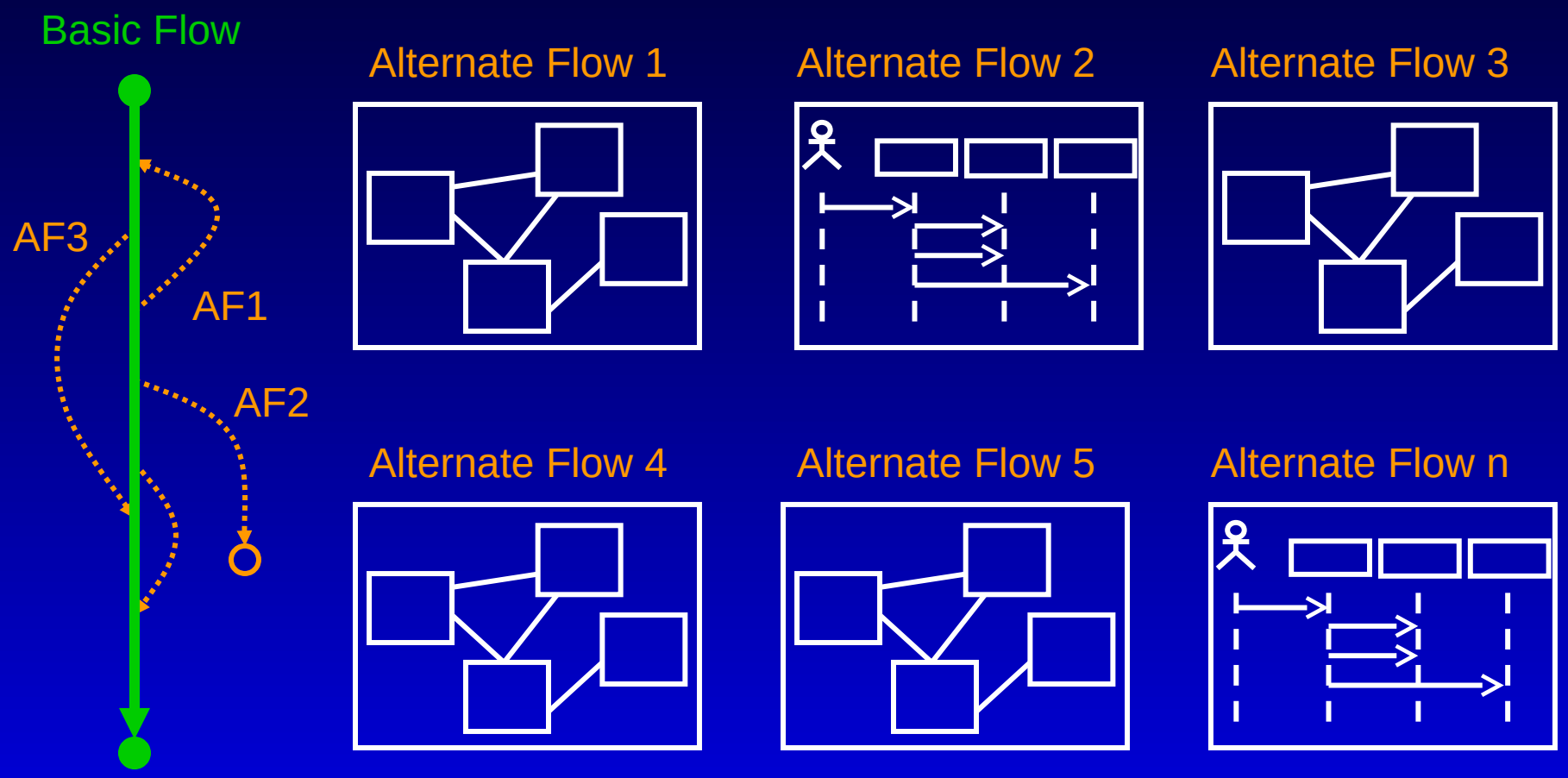
Example: Collaboration Diagram



Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

One Interaction Diagram Not Good Enough



Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Use-Case Analysis Steps

- Supplement the Use-Case Descriptions
- For each use-case realization
  - Find Classes from Use-Case Behavior
  - Distribute Use-Case Behavior to Classes
- For each resulting analysis class
  - Describe Responsibilities
  - Describe Attributes and Associations
  - Qualify Analysis Mechanisms
- Unify Analysis Classes
- Checkpoints

Copyright © 1998-1999 Rational Software, all rights reserved

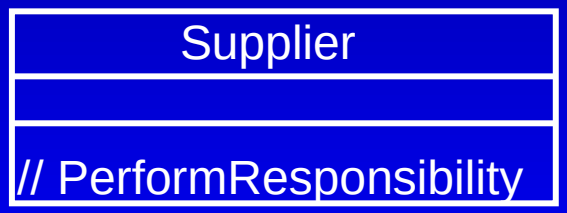
Rational unifying software teams

Describe Responsibilities

- What are responsibilities?
- How do I find them?

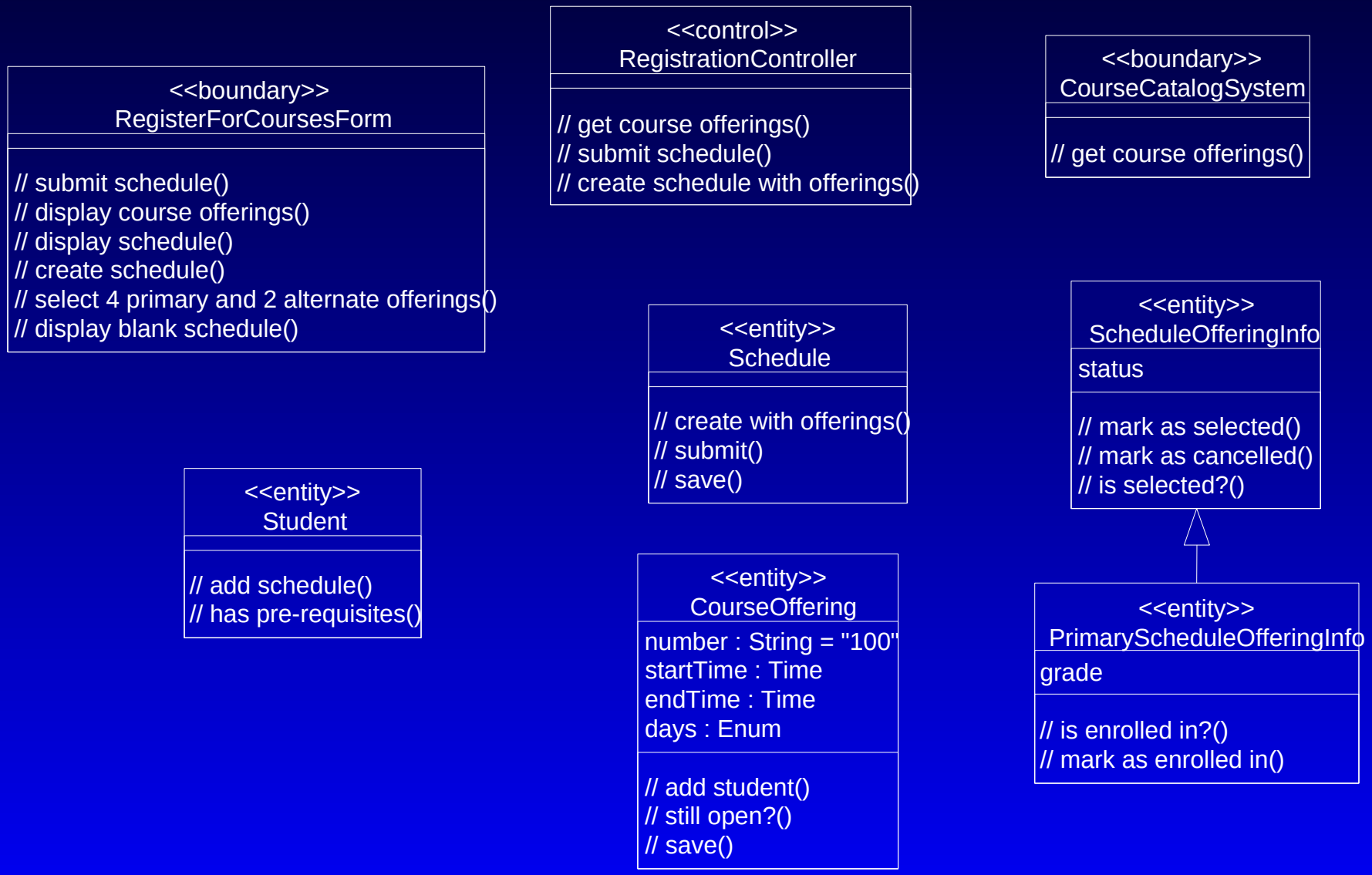
Interaction Diagram

Class Diagram



Copyright © 1998-1999 Rational Software, all rights reserved

Example: View of Participating Classes (VOPC) Class Diagram



Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Use-Case Analysis Steps

- Supplement the Use-Case Descriptions
- For each use-case realization
  - Find Classes from Use-Case Behavior
  - Distribute Use-Case Behavior to Classes
- For each resulting analysis class
  - Describe Responsibilities
  - ★ Describe Attributes and Associations
  - Qualify Analysis Mechanisms
- Unify Analysis Classes
- Checkpoints

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

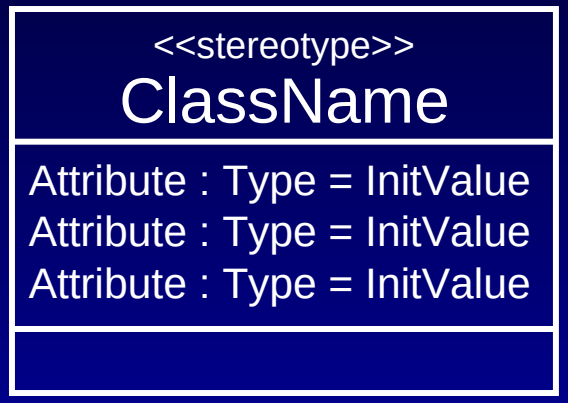
Describe Attributes and Associations

- Define Attributes
- Establish Aggregations and Associations

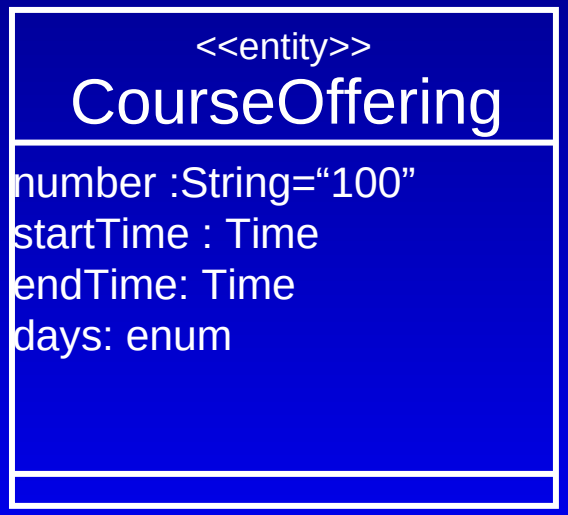
Copyright © 1998-1999 Rational Software, all rights reserved

Rational
unifying software teams

Review: What is an Attribute?



In analysis, do not spend time on attribute signatures



attribute

Copyright © 1998-1999 Rational Software, all rights reserved

Finding Attributes

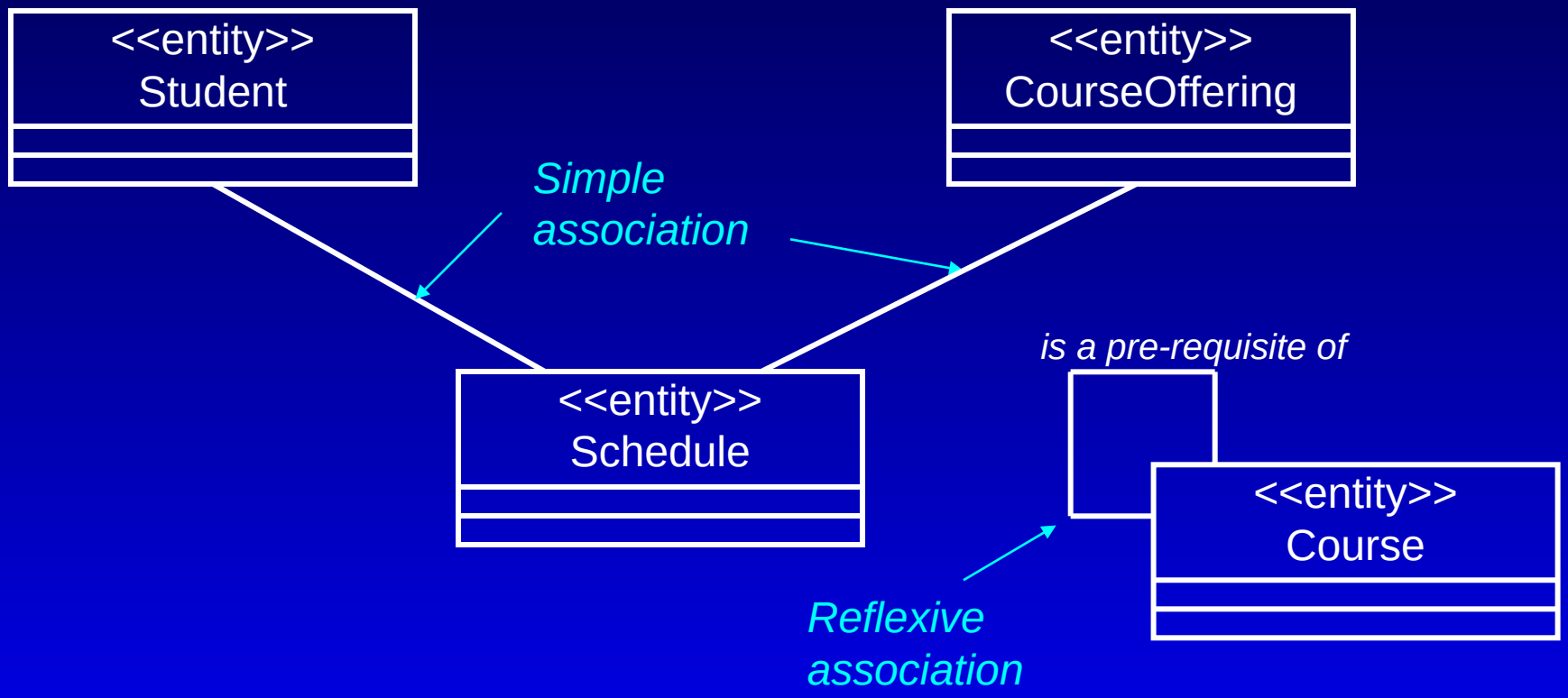
- Properties/characteristics of identified classes
- Information retained by identified classes
- “Nouns” that did not become classes
  - Information whose value is the important thing
  - Information that is uniquely "owned” by an object
  - Information that has no behavior

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Review: What is an Association?

- Models a semantic connection among instances



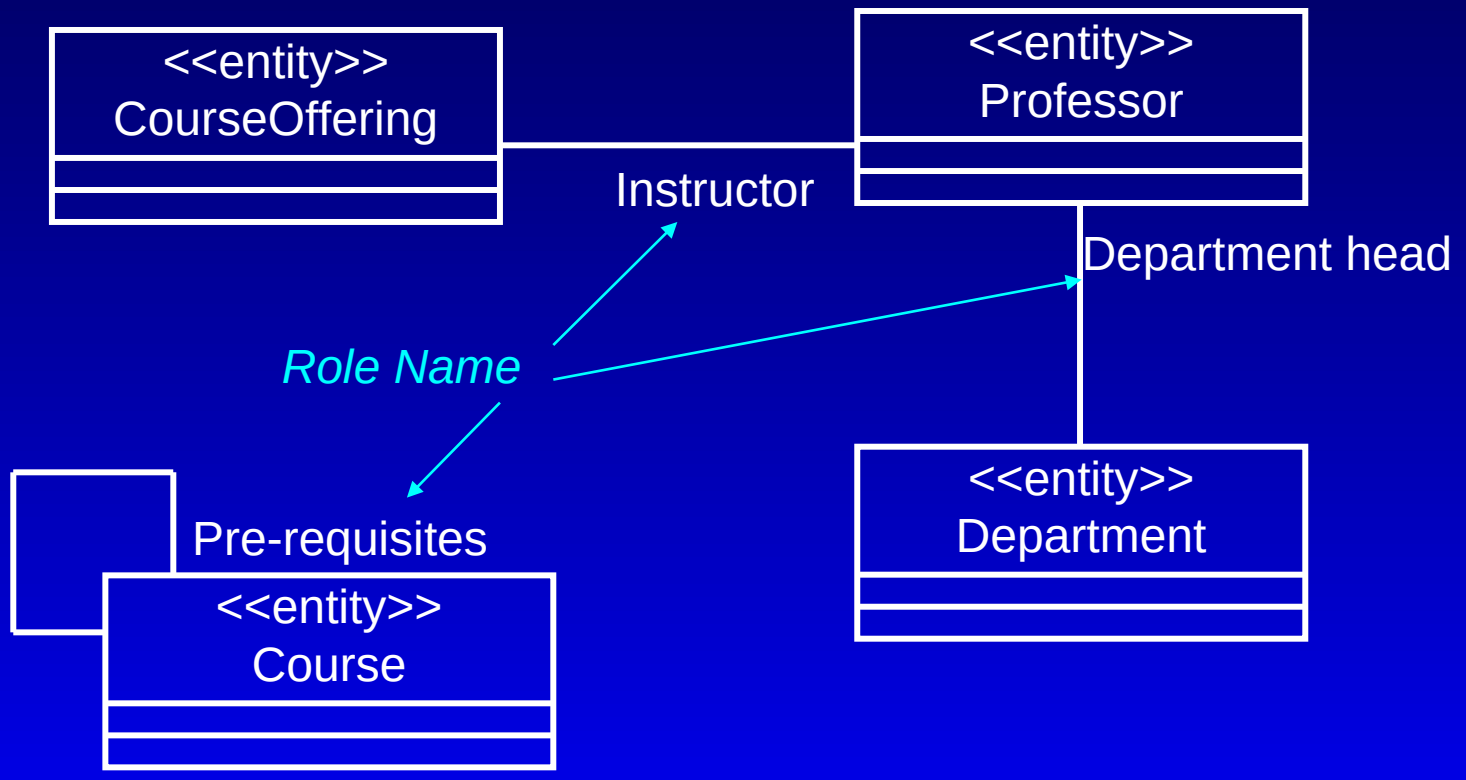
Association is a structural relationship

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

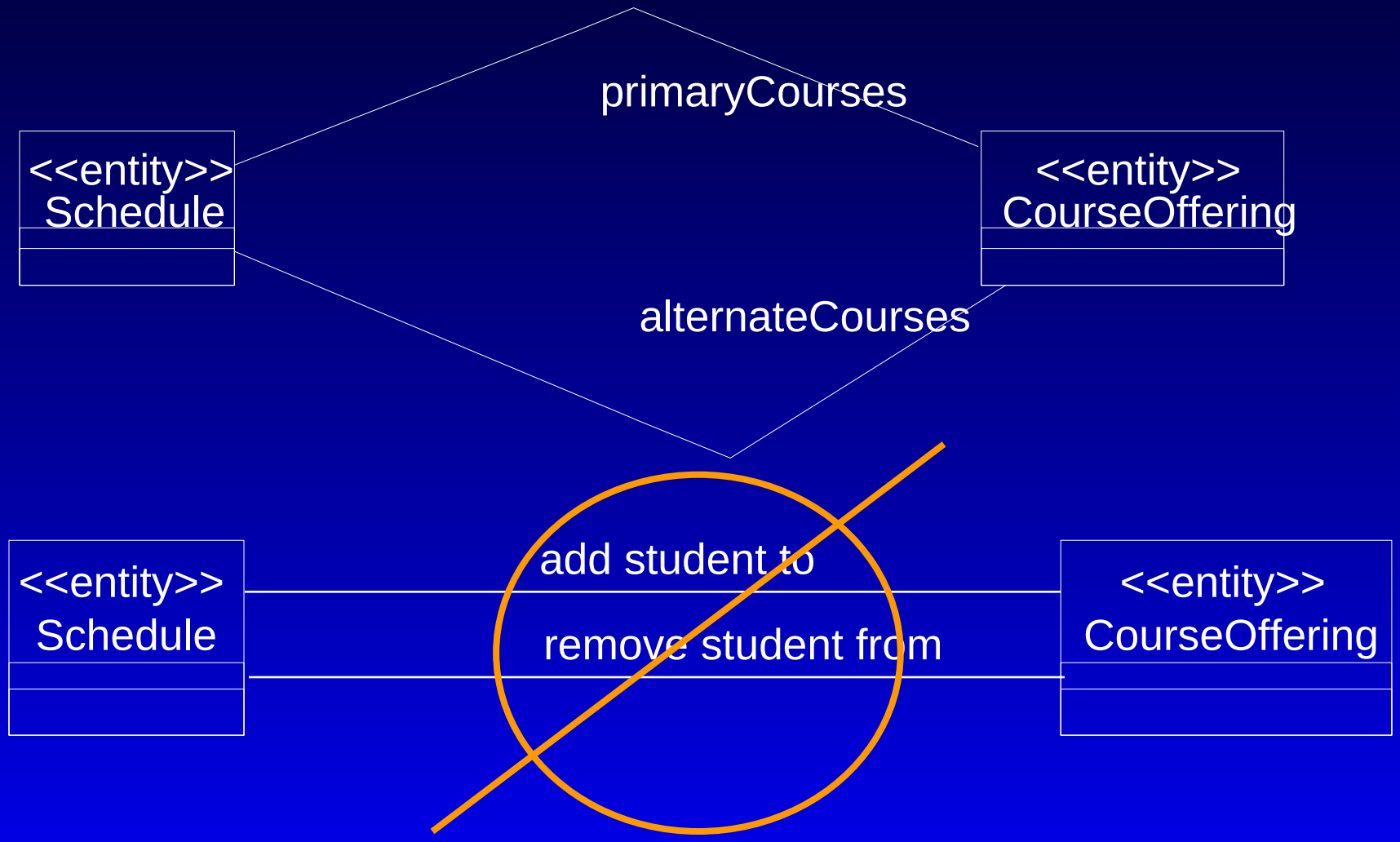
Review: What are Roles?

- The “face” that a class plays in the association



Copyright © 1998-1999 Rational Software, all rights reserved

Example: Multiple Associations



Multiple associations must reflect multiple roles

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Review: Multiplicity

- Unspecified
- Exactly one
- Zero or more (many, unlimited)
- One or more
- Zero or one (optional scalar role)
- Specified range
- Multiple, disjoint ranges

1

0..*

*

1..*

0..1

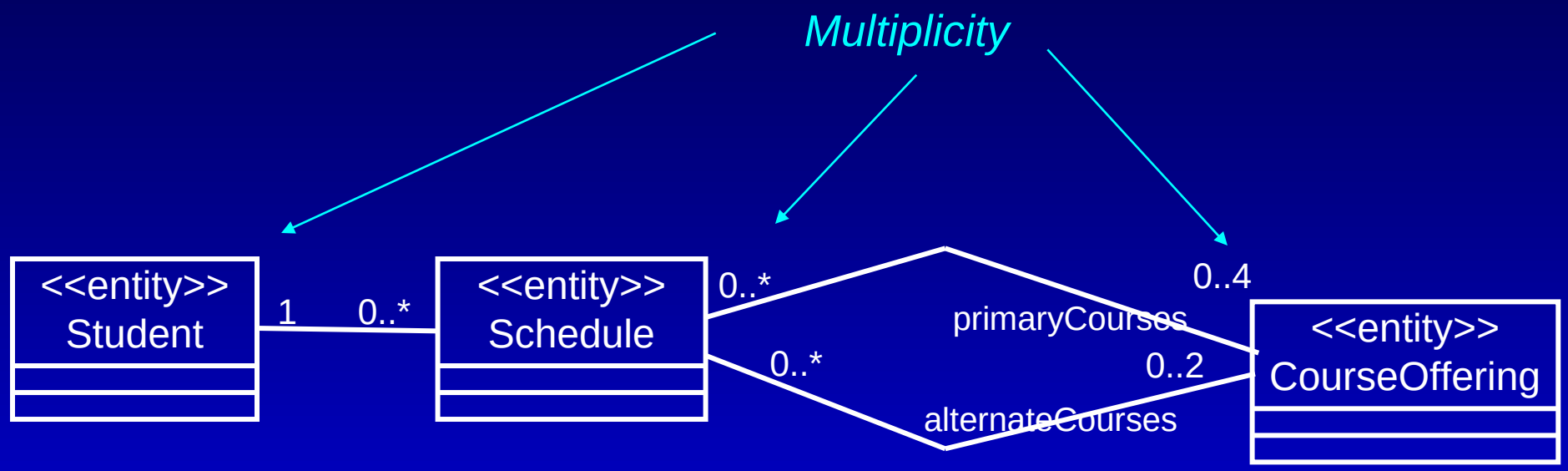
2..4

2, 4..6

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Example: Multiplicity

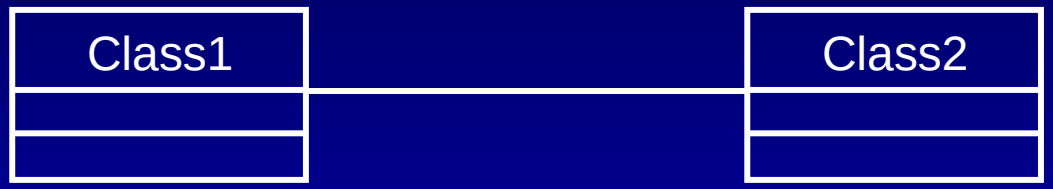


Copyright © 1998-1999 Rational Software, all rights reserved

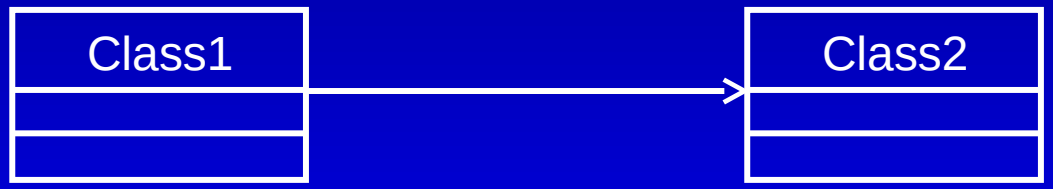
Review: Navigability

- Possible to navigate from an associating class to the target class

Bi-directional



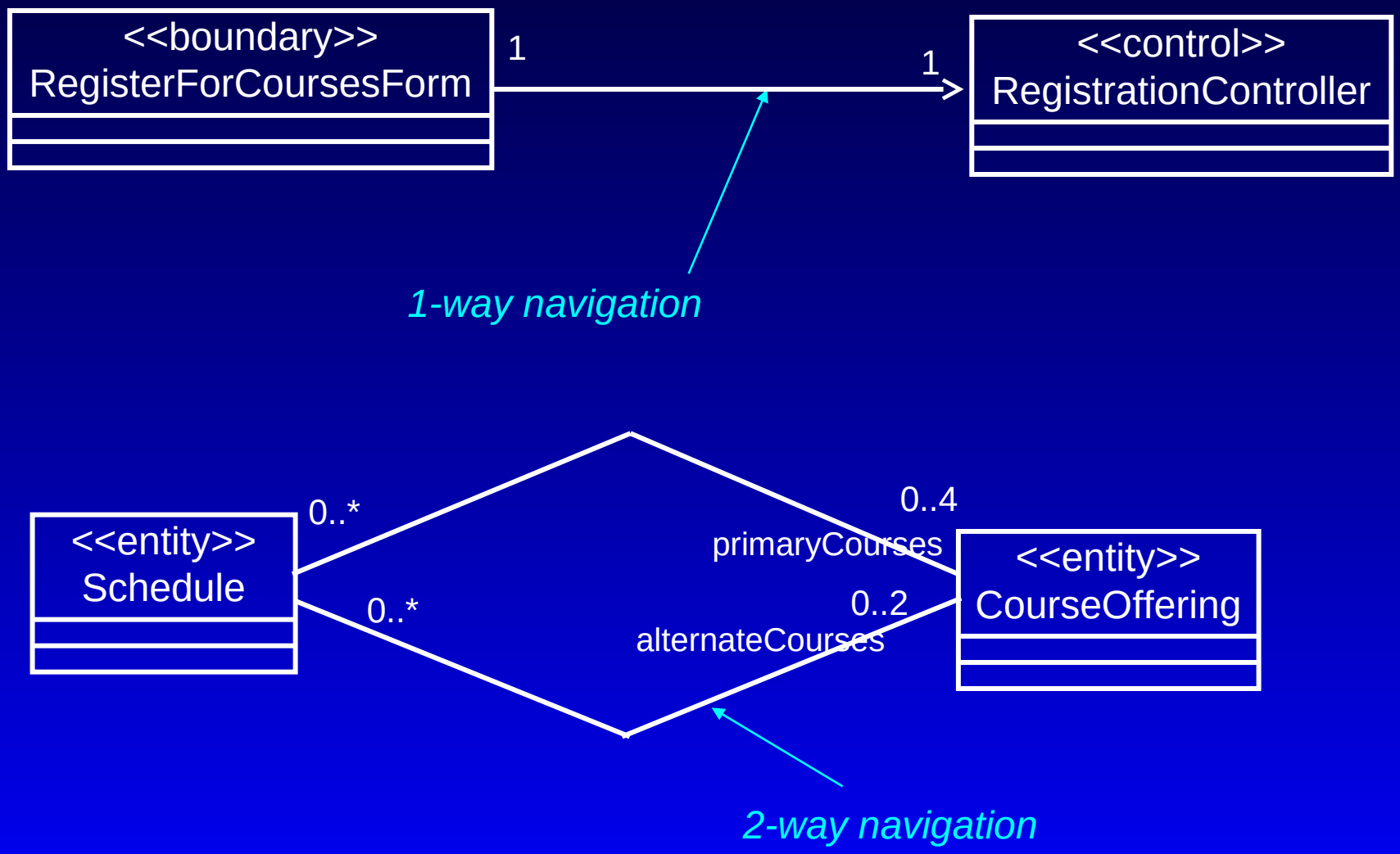
Uni-directional



Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Example: Navigability

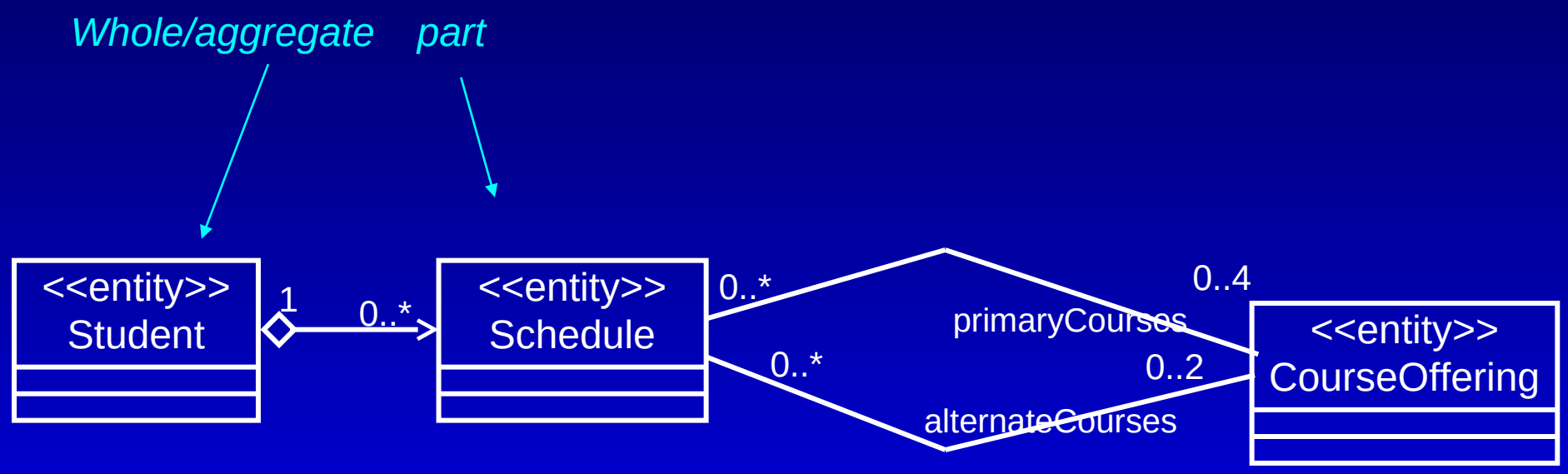


Copyright © 1998-1999 Rational Software, all rights reserved

Rational
unifying software teams

Review: What is Aggregation?

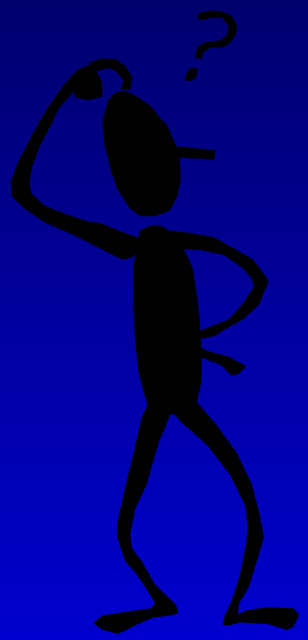
- A special form of association that models a whole-part relationship between an aggregate (the whole) and its parts

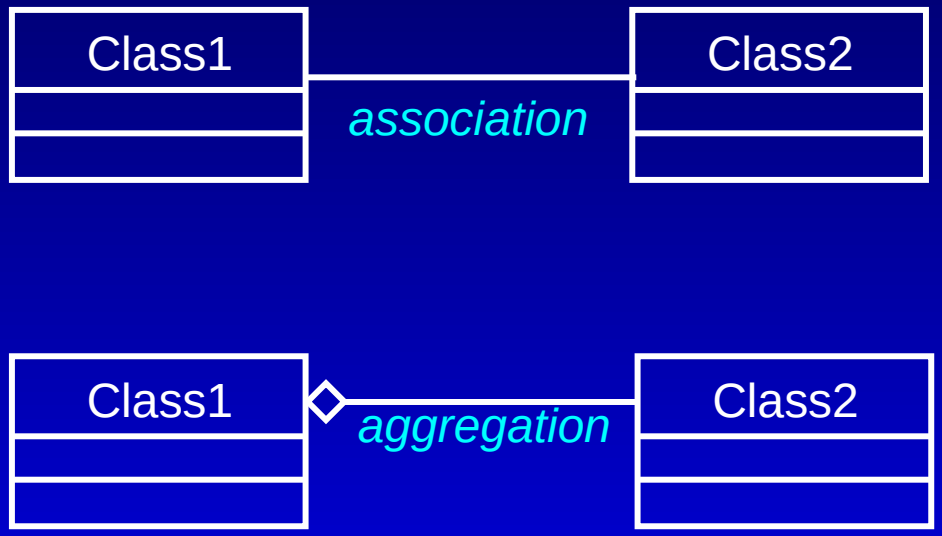


Copyright © 1998-1999 Rational Software, all rights reserved

Association or Aggregation?

- Consideration
  - Context, independent identity of Class2





When in doubt use association

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Finding Relationships

Collaboration Diagram

1: PerformResponsibility

:Client

:Supplier

Link

Client

Supplier

Class Diagram

Client

0..*

0..*

Prime suppliers

Supplier

PerformResponsibility()

Association

Relationship for every link!

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

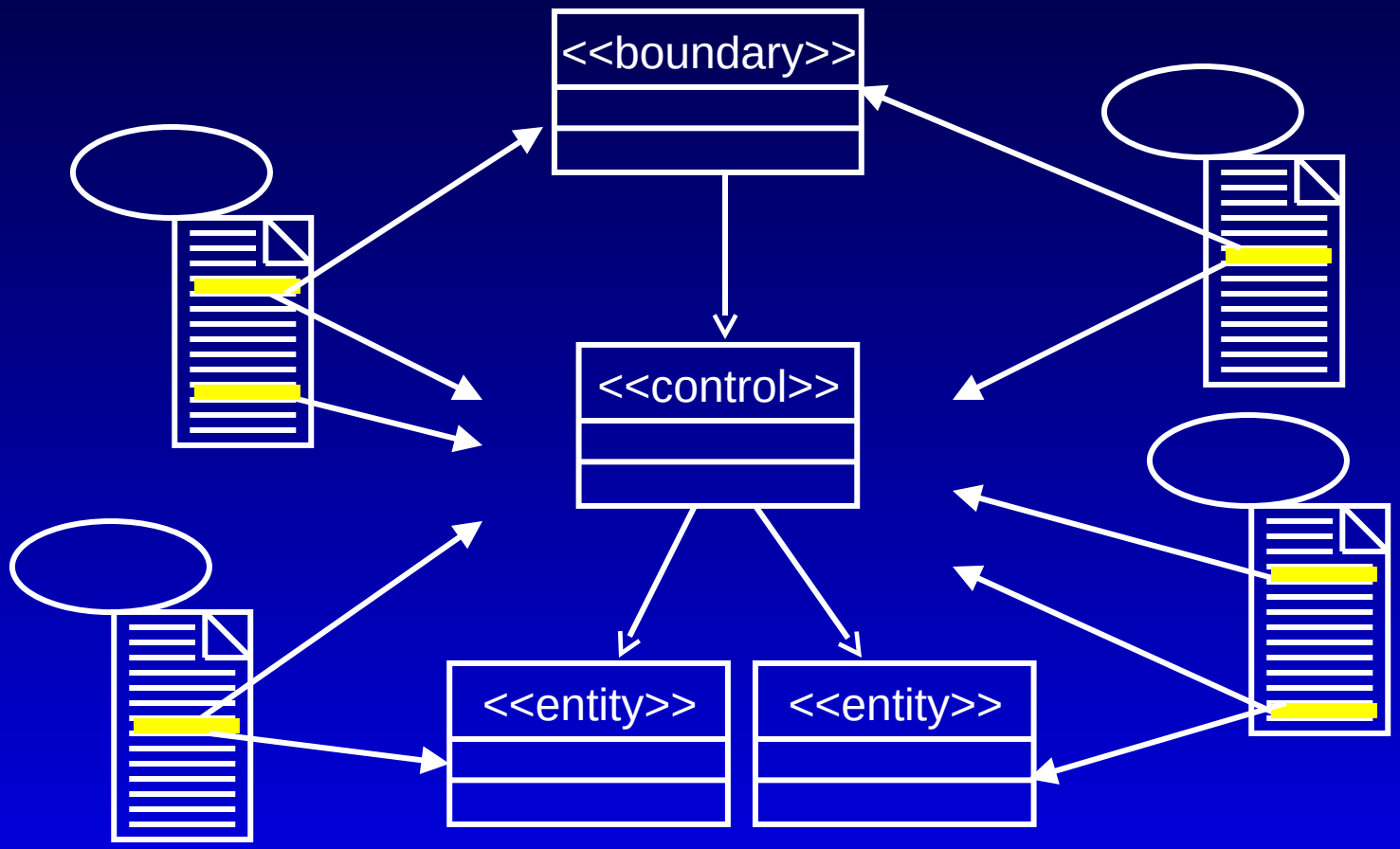
Use-Case Analysis Steps

- Supplement the Use-Case Descriptions
- For each use-case realization
  - Find Classes from Use-Case Behavior
  - Distribute Use-Case Behavior to Classes
- For each resulting analysis class
  - Describe Responsibilities
  - Describe Attributes and Associations
  - Qualify Analysis Mechanisms
- Unify Analysis Classes
- Checkpoints

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

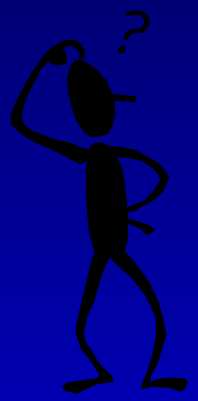
Unify Analysis Classes

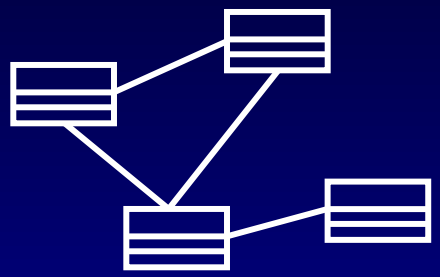


Copyright © 1998-1999 Rational Software, all rights reserved

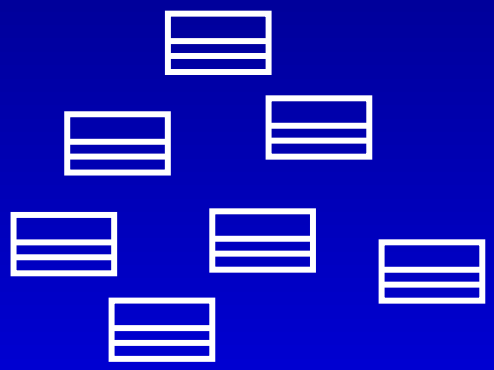
Rational
unifying software teams

Evaluate Your Results

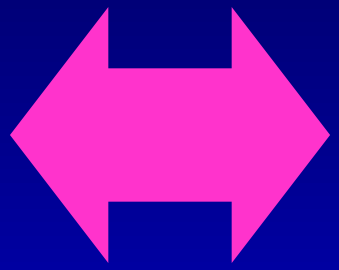




Design Model

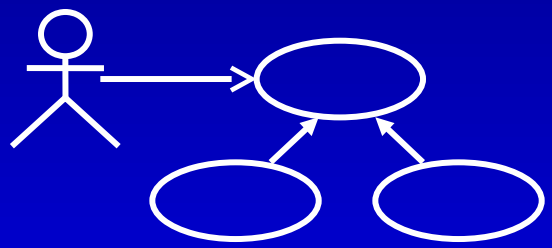


Analysis Classes



Supplementary Specification

Glossary



Use-Case Model

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Use-Case Analysis Steps

- Supplement the Use-Case Descriptions
- For each use-case realization
  - Find Classes from Use-Case Behavior
  - Distribute Use-Case Behavior to Classes
- For each resulting analysis class
  - Describe Responsibilities
  - Describe Attributes and Associations
  - Qualify Analysis Mechanisms
- Unify Analysis Classes
- Checkpoints

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Checkpoints: Analysis Classes

- Are the classes reasonable?
- Does the name of each class clearly reflect the role it plays?
- Does the class represent a single well-defined abstraction?
- Are all attributes and responsibilities functionally coupled?
- Does the class offer the required behavior?
- Are all specific requirements on the class addressed?

(continued)

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Checkpoints: Use-Case Realizations

- Have all the main and/or sub-flows been handled, including exceptional cases?
- Have all the required objects been found?
- Has all behavior been unambiguously distributed to the participating objects?
- Has behavior been distributed to the right objects?
- Where there are several interaction diagrams, are their relationships clear and consistent?

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams

Review: Use-Case Analysis

- What is the purpose of Use-Case Analysis?
- What is an analysis class? Name and describe the three analysis stereotypes.
- What is a use-case realization?
- Describe some considerations when allocating responsibilities to analysis classes.
- How many interaction diagrams should be produced during Use-Case Analysis?

Copyright © 1998-1999 Rational Software, all rights reserved

Rational unifying software teams